

VISHNU INSTITUTE OF TECHNOLOGY(AUTONOMOUS)
(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)
Vishnupur, BHIMAVARAM – 534 202

DEPARTMENT: Computer Science & Engineering

Certificate

*Certified that this is a bonafide record of practical work done by
.....Regd. No: of
B.Tech Semester in the Laboratory during the year
2025–2026.*

Date

Staff Member/In-Charge

Head of the Department

Submitted for the Practical Examination held on:

Internal Examiner

External Examiner

VISHNU INSTITUTE OF TECHNOLOGY(AUTONOMOUS)
(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)
Vishnupur, BHIMAVARAM – 534 202

INDEX

S.No	Date	Experiment	Page	Marks

VISHNU INSTITUTE OF TECHNOLOGY(AUTONOMOUS)
(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)
Vishnupur, BHIMAVARAM – 534 202

INDEX

S.No	Date	Experiment	Page	Marks

Exp 1 Lab on web services

To develop and deploy a simple web service using Python Flask framework and access it through a web browser.

Software Requirements

- Windows 10/11/Ubuntu
- Python 3.x
- Flask Library
- Web Browser (Chrome/Edge)

A web service is a software system designed to support communication between applications over a network using HTTP protocol. Flask is a lightweight web framework in Python that allows us to create web services easily.

Procedure

1. Install Python software.
2. Install Flask library using pip command.
3. Create a Python file named webservice.py.
4. Write Flask web service code.
5. Run the program using command prompt.
6. Open browser and access the URLs.
7. Observe the output.

Program

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return "Hello! This is my first Cloud Web Service"
@app.route('/student')
def student():
    return "Welcome Student to Cloud Lab"
if __name__ == '__main__':
    app.run(debug=True)
```

Output

When the server runs:

Running on `http://127.0.0.1:5000/`

Browser Output:

URL 1:

`http://127.0.0.1:5000/`

Output:

Hello! This is my first Cloud Web Service

URL 2:

`http://127.0.0.1:5000/student`

Output:

Welcome Student to Cloud Lab

Exp 2 — IPC, Messaging, Publish/Subscribe

To demonstrate Inter Process Communication (IPC) using socket programming and implement a simple messaging system between client and server.

Software Requirements

- Windows 10/11/Ubuntu
- Python 3.x
- Command Prompt

Inter Process Communication (IPC) is a mechanism that allows processes to communicate with each other and exchange data. Messaging is a form of IPC where processes send and receive messages. Socket programming enables communication between two systems over a network using IP addresses and port numbers.

The Publish/Subscribe model is a messaging paradigm where publishers send messages without knowing the subscribers, and subscribers receive messages based on topics of interest.

Procedure

1. Install Python software.
2. Create a server program file named server.py.
3. Create a client program file named client.py.
4. Write socket programming code for both client and server.
5. Open two command prompts.
6. Run server program first.
7. Run client program next.
8. Observe the message exchange between client and server.

Program

Server Program

```
import socket
```

```
s = socket.socket()
s.bind(('localhost', 5000))
s.listen(1)
```

```
print("Server waiting for connection...")
```

```
conn, addr = s.accept()
print("Connected from", addr)
```

```
message = conn.recv(1024).decode()
print("Client says:", message)

conn.send("Message received successfully".encode())

conn.close()
```

Client Program

```
import socket

c = socket.socket()
c.connect(('localhost', 5000))

c.send("Hello Server".encode())

reply = c.recv(1024).decode()
print("Server reply:", reply)

c.close()
```

Output

Server Side Output

```
Server waiting for connection...
Connected from ('127.0.0.1', 12345)
Client says: Hello Server
```

Client Side Output

```
Server reply: Message received successfully
```

Exp 3 — Installation of VirtualBox and Creation of Virtual Machine

Aim

To install Oracle VirtualBox on a Windows system and create a virtual machine by installing a Linux operating system.

Software Requirements

- Windows 10/11 (64-bit)
- Oracle VirtualBox
- Ubuntu Linux ISO file
- Minimum 4 GB RAM (Recommended 8 GB)

Virtualization is the process of creating a virtual version of computing resources such as hardware platforms, storage devices, and operating systems. A virtual machine (VM) is a software-based environment that behaves like a physical computer. Oracle VirtualBox is a type-2 hypervisor that allows multiple operating systems to run on a single physical machine.

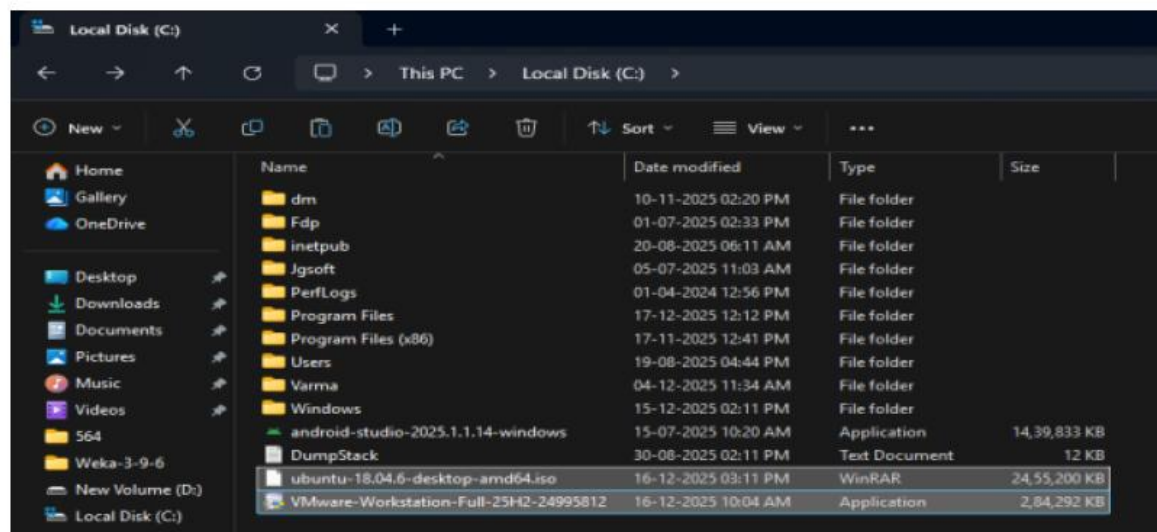
Procedure

1. Download Oracle VirtualBox from the official website.
2. Install VirtualBox on the Windows system.
3. Download Ubuntu Linux ISO file from the official Ubuntu website.
4. Open VirtualBox and click on **New** to create a virtual machine.
5. Provide the name, type (Linux), and version (Ubuntu 64-bit).
6. Allocate memory (minimum 2 GB RAM).
7. Create a virtual hard disk (minimum 20 GB).
8. Attach the Ubuntu ISO file to the virtual machine storage.
9. Start the virtual machine.
10. Install Ubuntu operating system by following on-screen instructions.
11. Verify the successful installation by viewing the Ubuntu desktop.

Phase 1: VMware Workstation Installation

Step 1: Software Source

Navigate to the directory containing your software files to find the VMware installer and the Ubuntu ISO image. Identify the executable file named VMware-Workstation-Full which is required to start the host software installation.



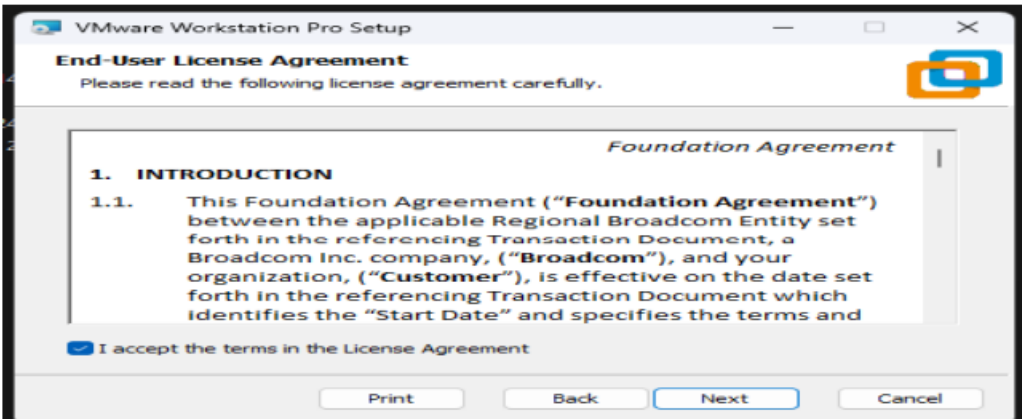
Step 2: Initialization

Double-click the installer to initiate the extraction process and prepare the system for the Setup Wizard. Ensure you have administrative privileges to allow the installer to make necessary system changes.



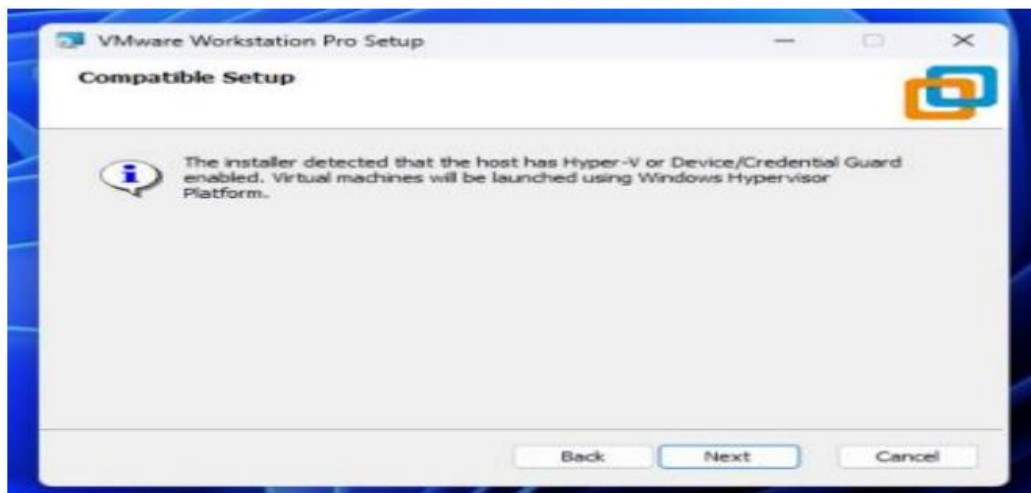
Step 3: Welcome Wizard

The VMware Workstation Pro Setup Wizard will launch, providing a guided interface for the installation. Click the Next button to begin the configuration of the virtualization software on your host machine.



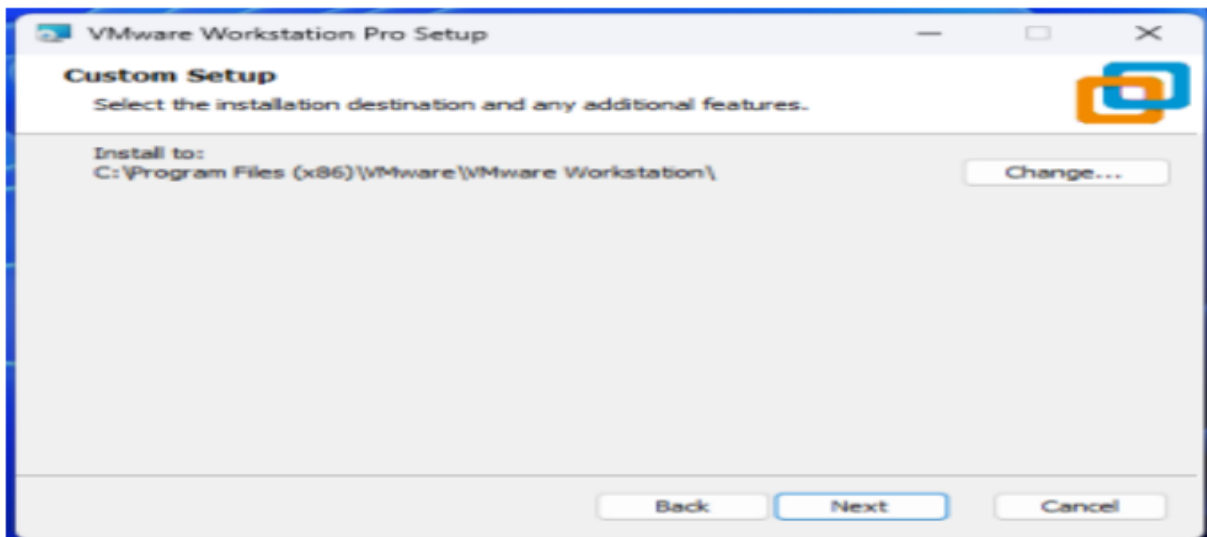
Step 4: End-User License Agreement

Review the terms and conditions provided in the Foundation Agreement regarding the use of Broadcom/VMware software. Check the box to accept the license terms and click Next to proceed with the installation.



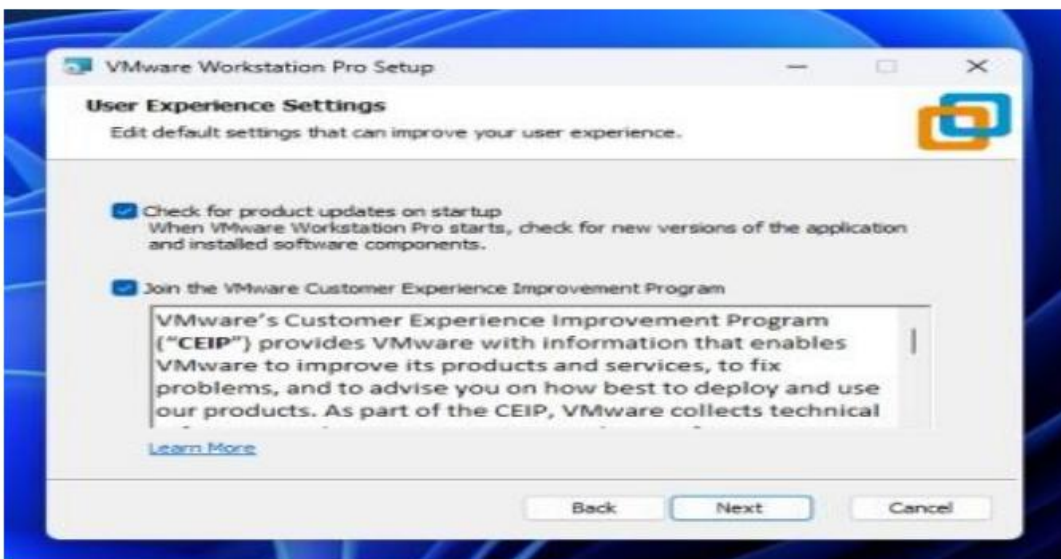
Step 5: System Compatibility

The installer checks for compatibility with Windows features like Hyper-V or Device/Credential Guard. Click Next to acknowledge that virtual machines will be launched using the Windows Hypervisor Platform.



Step 6: Custom Setup

Choose the destination folder where the VMware Workstation files will be installed on your local disk. You may click Change to select a different directory or click Next to accept the default program files path.

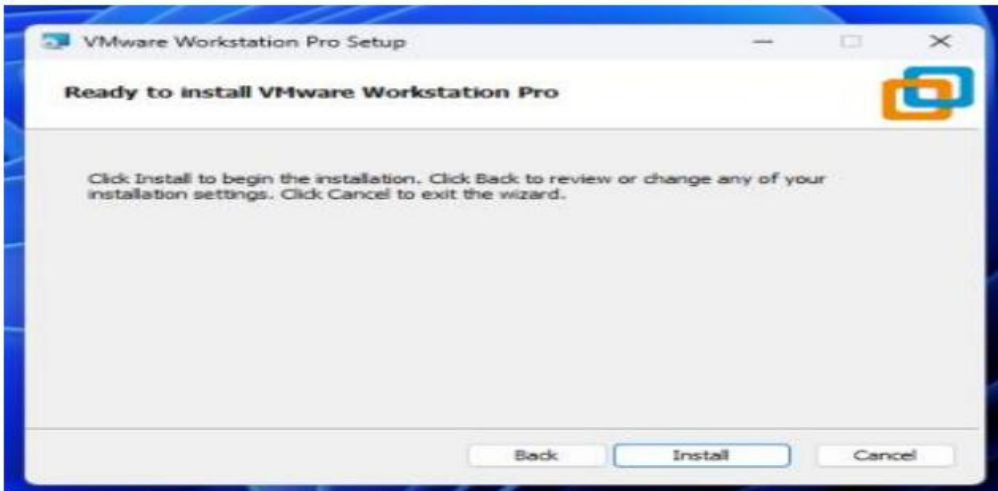


Step 7: User Experience Settings

Configure whether the application should check for product updates on startup for improved security. You can also choose to join the Customer Experience Improvement Program to send anonymous data to VMware.

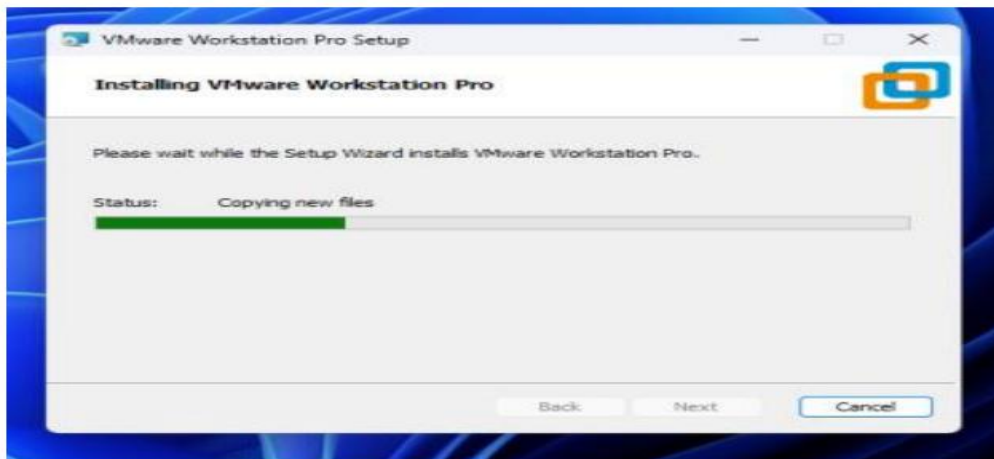
Step 8: Shortcut Creation

Select the locations where you want the installation wizard to place program shortcuts for easy access. It is recommended to keep both the Desktop and Start Menu Programs Folder options checked.



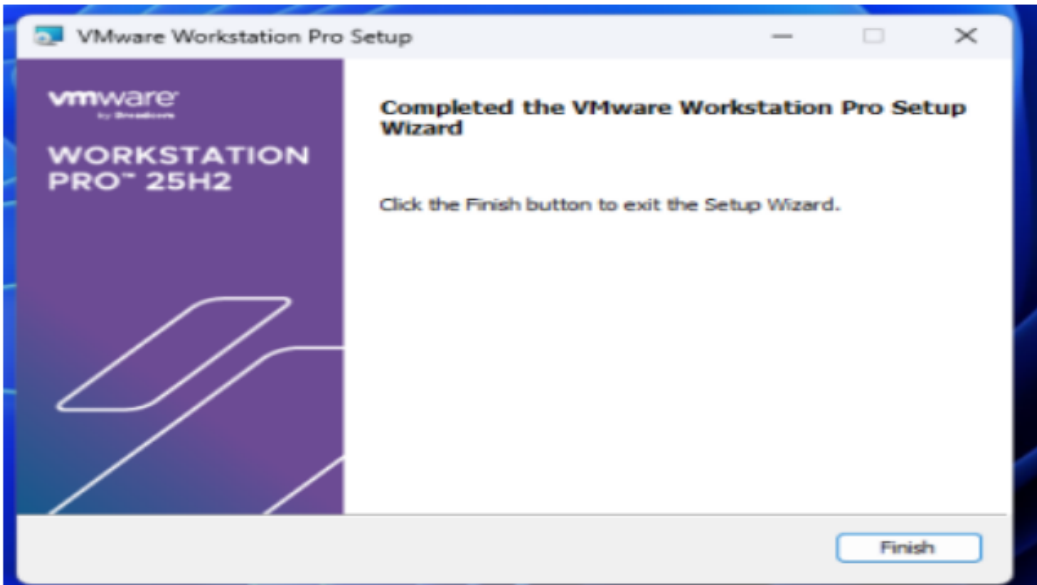
Step 9: Ready to Install

Review your installation settings one last time before the software files are written to the disk. Click the Install button to begin the actual deployment of VMware Workstation Pro.



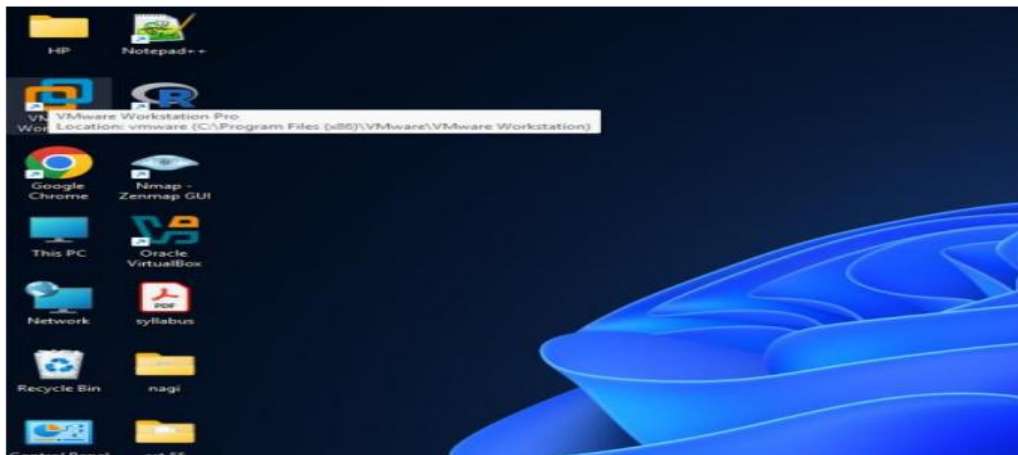
Step 10: Installation Progress

The wizard displays a progress bar while it copies new files and registers the virtualization services. Avoid interrupting this process to ensure all drivers and network adapters are configured correctly.



Step 11: Completion

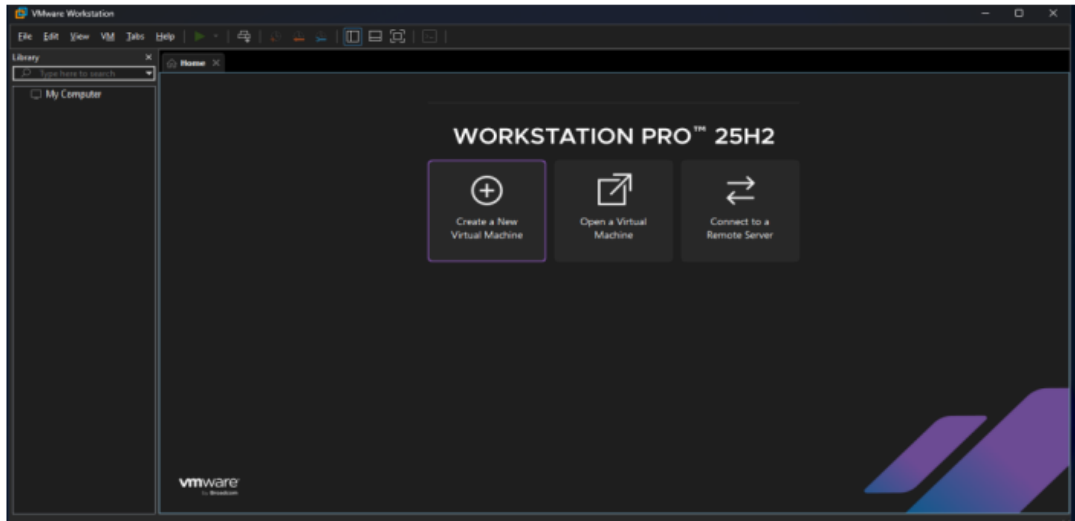
Once the "Completed the VMware Workstation Pro Setup Wizard" message appears, the software is ready. Click Finish to exit the installer and return to your desktop.



Phase 2: Virtual Machine Configuration

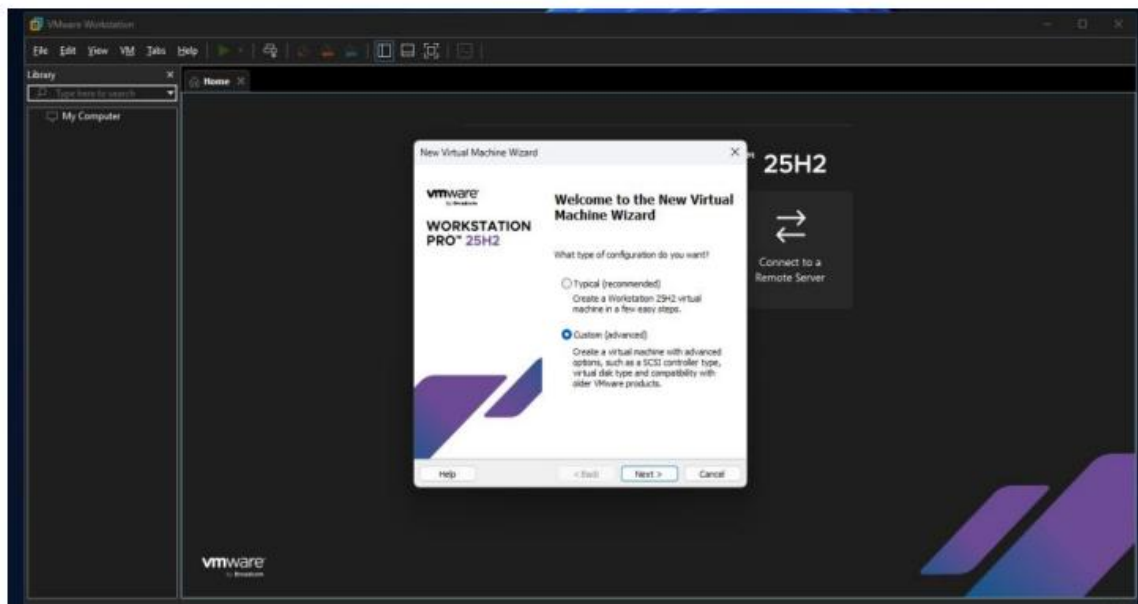
Step 12: Launching the Application

Locate the VMware Workstation Pro icon on your desktop and double-click it to open the main dashboard. This interface serves as the centralized hub for managing all your virtualized environments.



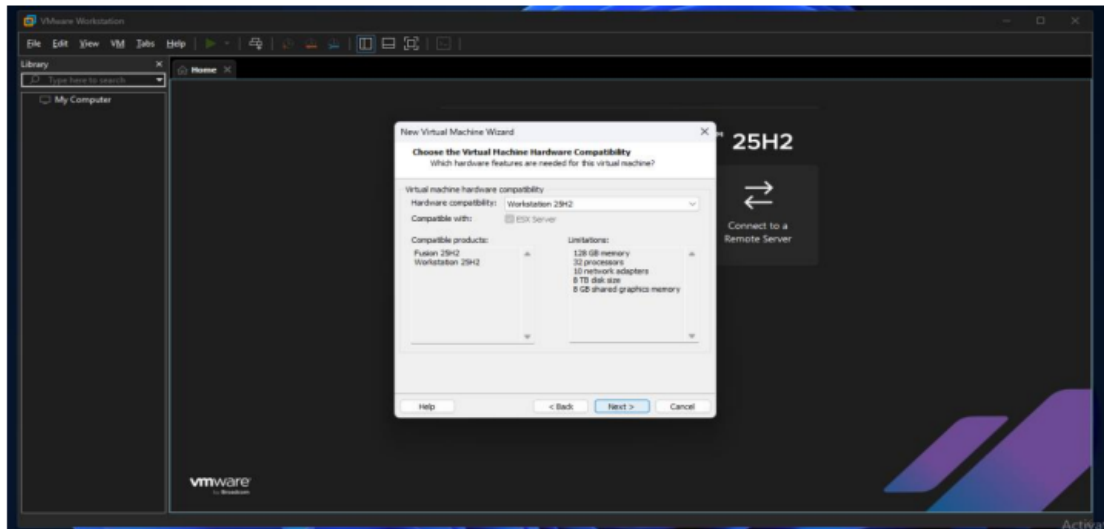
Step 13: New VM Wizard

From the home screen, click on Create a New Virtual Machine to start the guest OS setup process. This will open a new wizard specifically designed to define the hardware of your virtual computer.



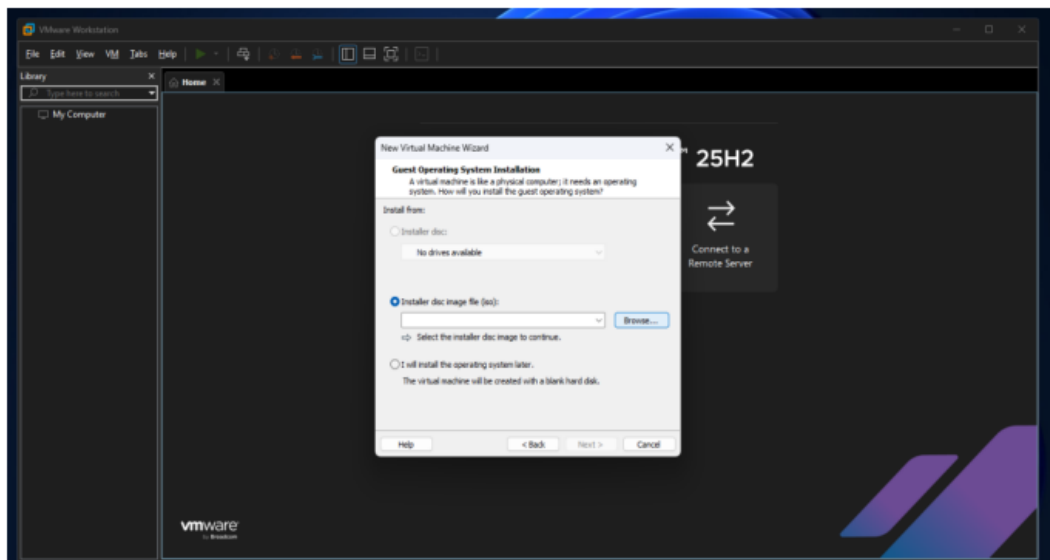
Step 14: Configuration Type

Select the Custom (advanced) configuration option to manually specify hardware limits and compatibility. This is useful for advanced users who need to tweak specific controller types or virtual hardware versions.



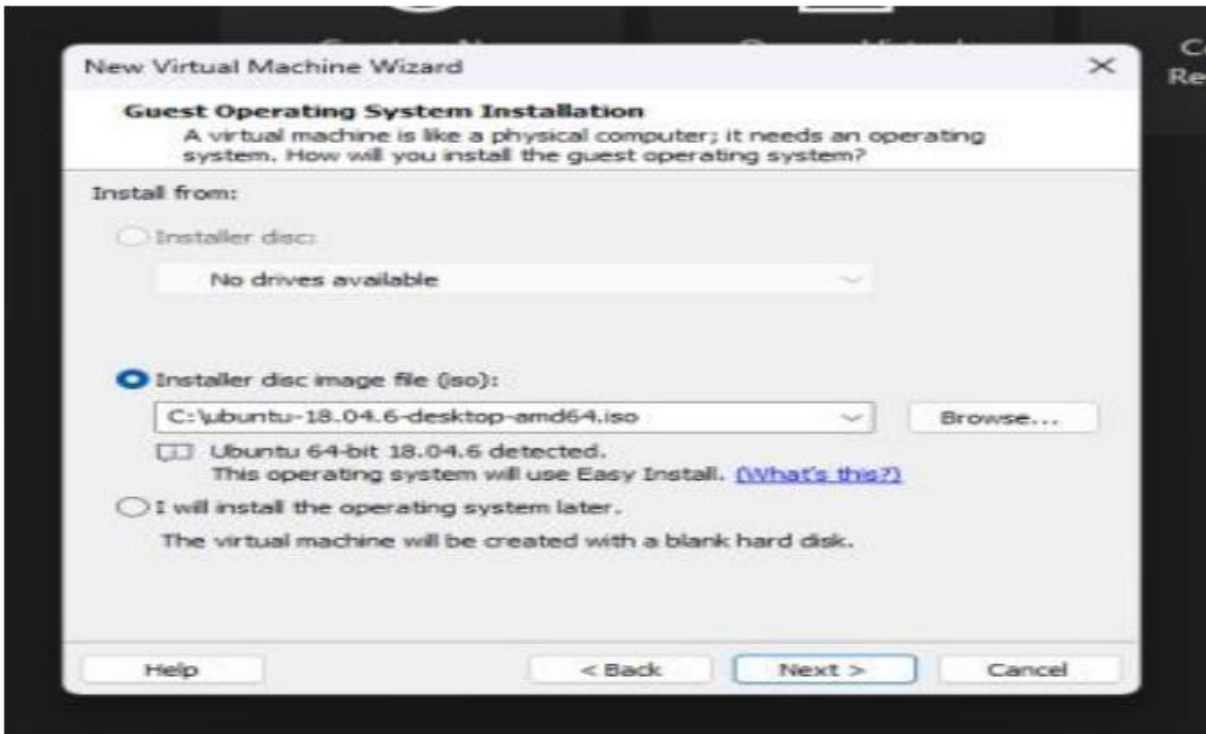
Step 15: Hardware Compatibility

Choose the Workstation hardware version that best suits your needs, typically the latest available version. This screen also displays the maximum limits for memory, processors, and disk size supported by this version.



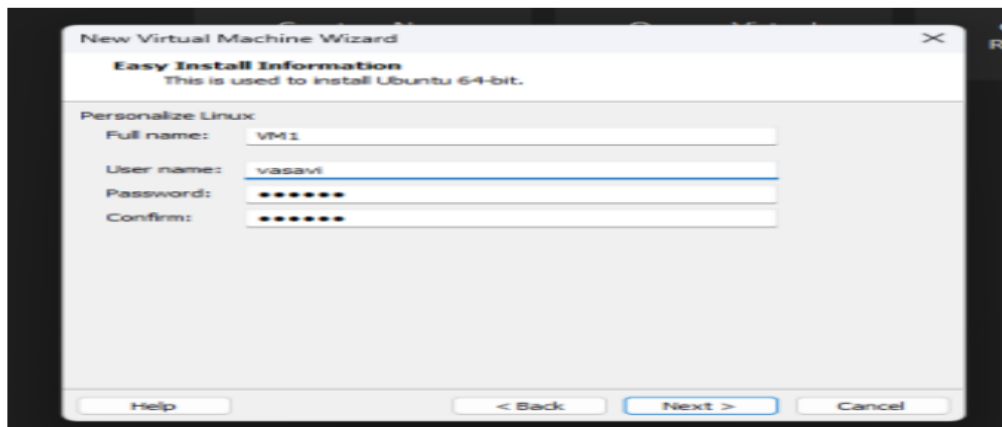
Step 16: Guest OS Source

Select the option to install from an Installer disc image file (iso) to mount your operating system file. This acts as a virtual CD/DVD drive that the virtual machine will boot from during the first start.



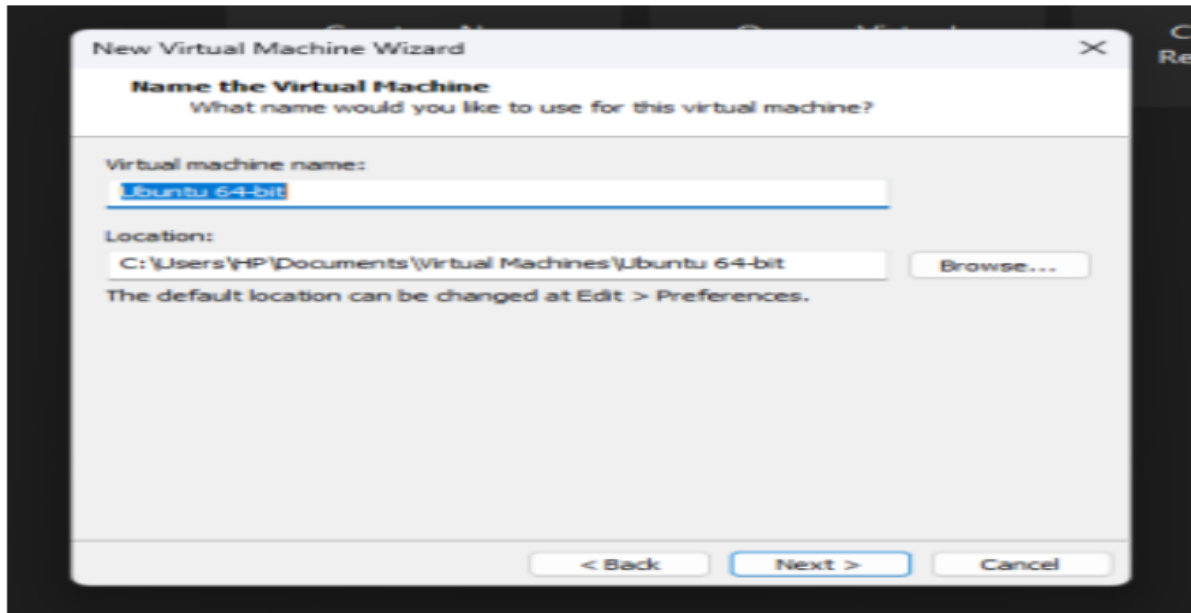
Step 17: ISO Selection

Browse your files to select the Ubuntu 18.04 ISO image you previously identified in the first step. VMware will automatically detect that an Ubuntu 64-bit operating system is present in the file.



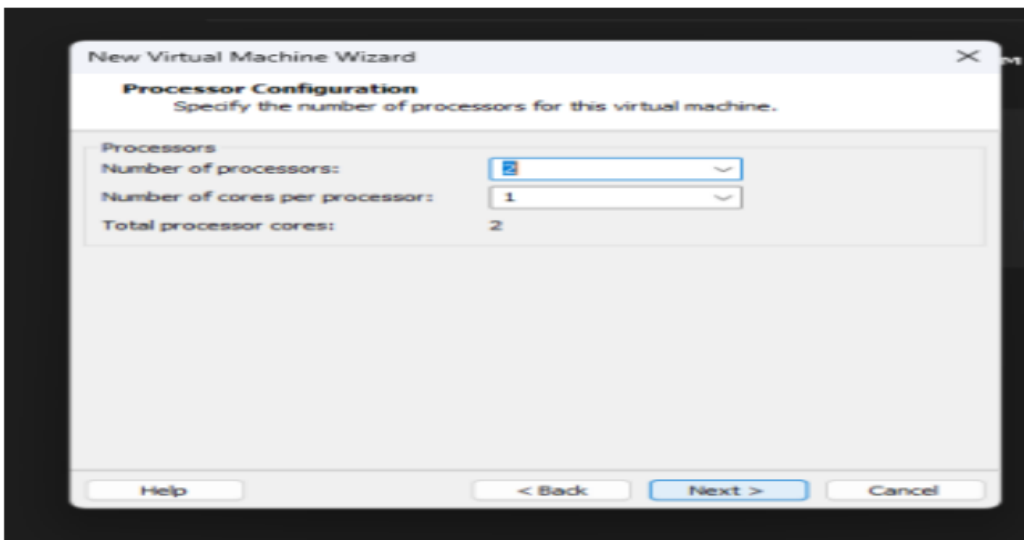
Step 18: Personalization

Enter the full name, username, and password that will be used for the primary account in the Linux environment. This "Easy Install" feature automates the account creation process within the Ubuntu installer.



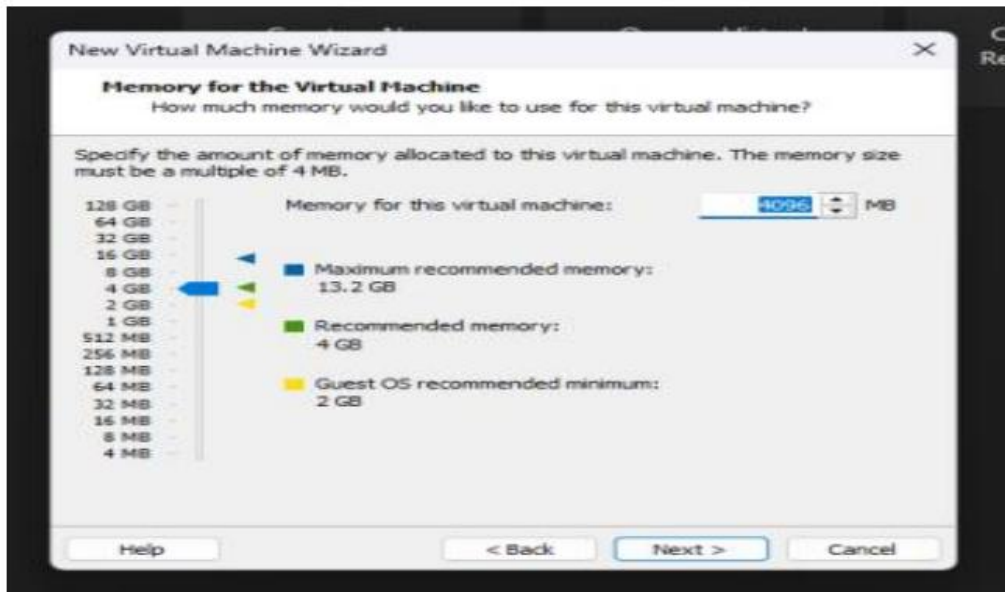
Step 19: Virtual Machine Name

Provide a descriptive name for your VM, such as "Ubuntu 64-bit," to identify it within the VMware library. You can also specify a custom location on your hard drive to store the large virtual disk files.



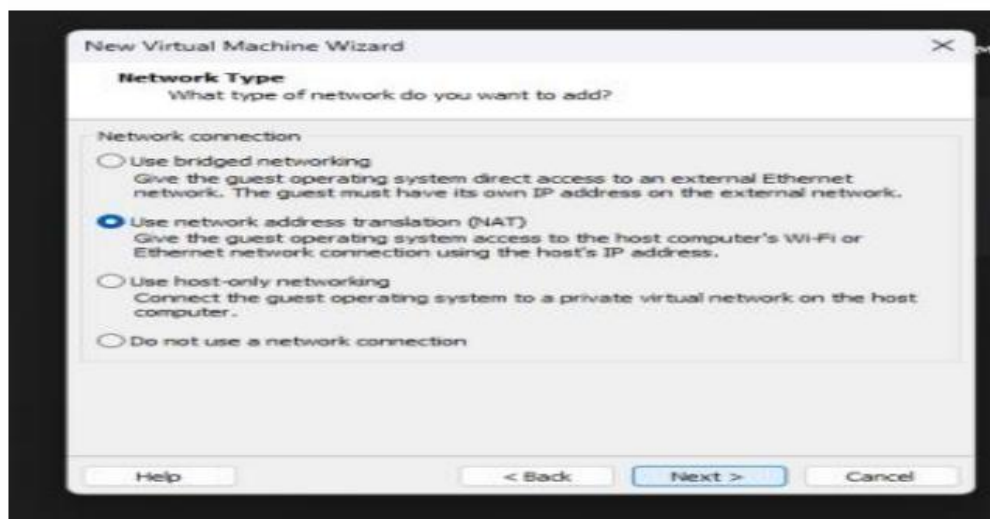
Step 20: Processor Allocation

Assign the number of processors and cores that the virtual machine is allowed to use from your host CPU. Allocating 2 cores is usually sufficient for a smooth experience with a Linux desktop environment.



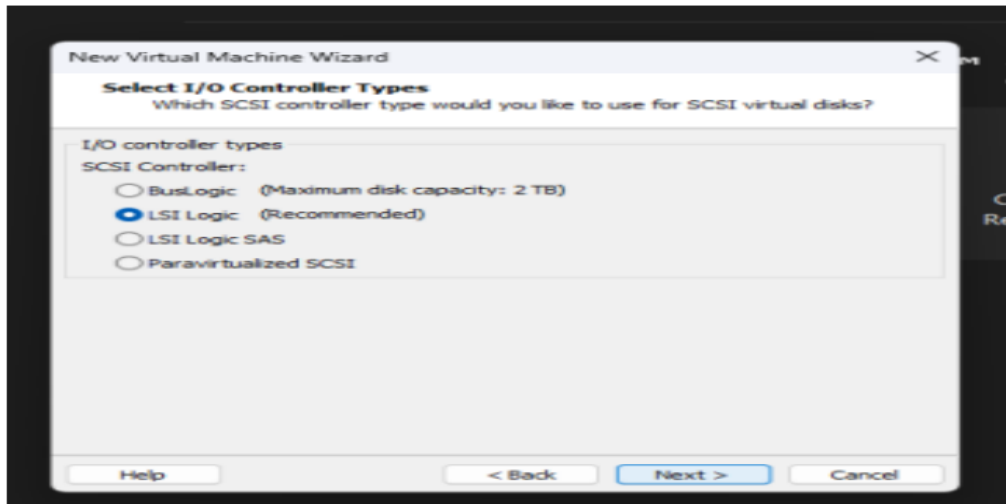
Step 21: Memory Allocation

Adjust the slider to allocate the amount of RAM for the virtual machine, with 4096 MB (4GB) being a balanced choice. Ensure you leave enough RAM for your host Windows OS to continue running smoothly.



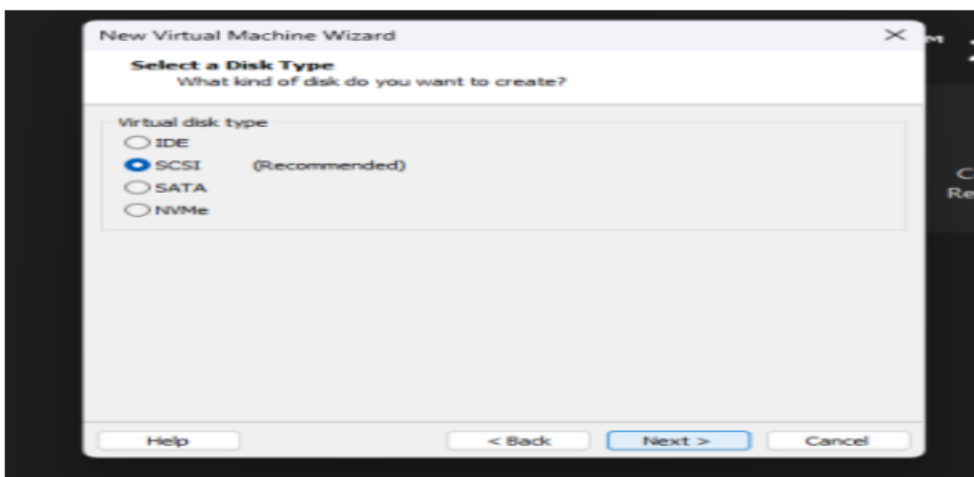
Step 22: Network Type

Select Network Address Translation (NAT) to allow the virtual machine to share the host's IP address and internet. This is the simplest way to give your Linux VM immediate access to web browsing and updates.



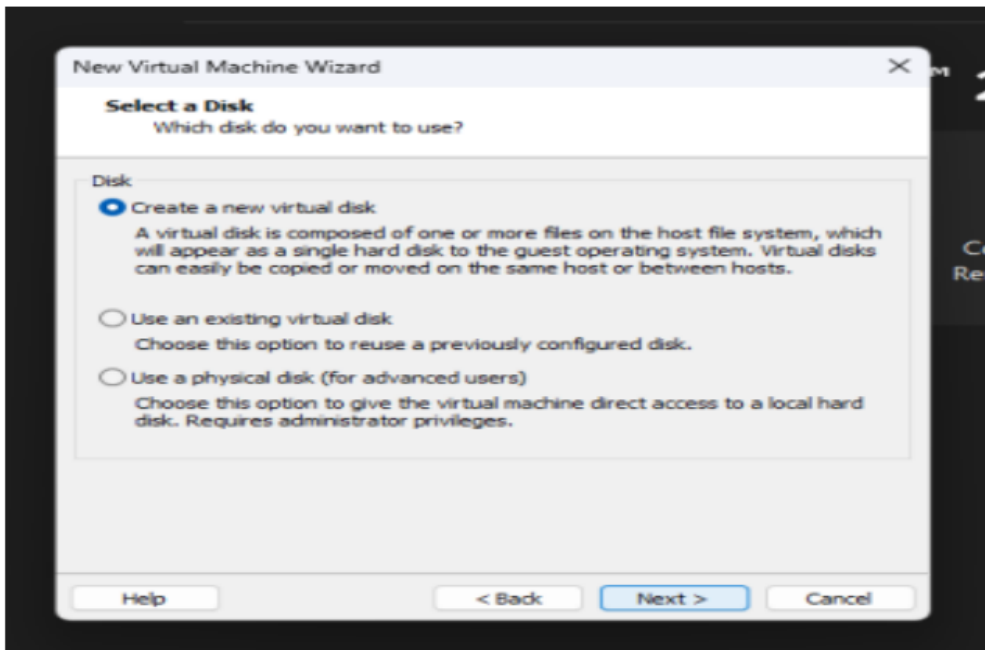
Step 23: I/O Controller Type

Choose the I/O controller type, with LSI Logic being the recommended default for compatibility. This setting determines how the virtual machine communicates with its virtual hard drive.



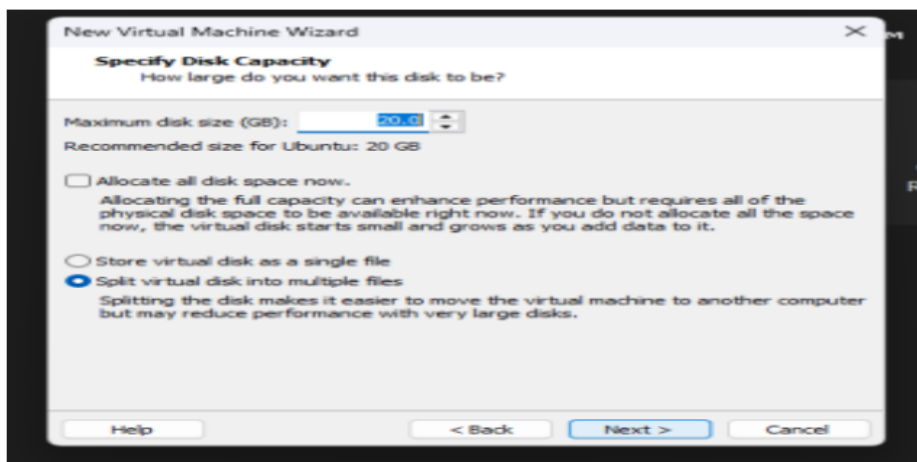
Step 24: Disk Type

Select SCSI as the virtual disk type, which is the recommended standard for performance in VMware. This defines the interface used by the virtual operating system to read and write data.



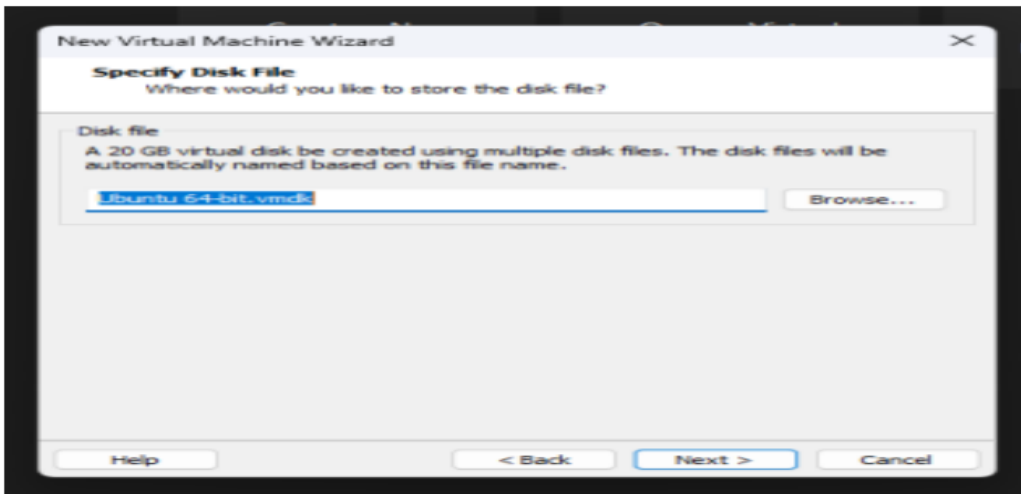
Step 25: Disk Selection

Choose Create a new virtual disk to allocate fresh space on your physical drive for the Linux installation. This creates a file on your host computer that acts as a hard drive for the guest OS.



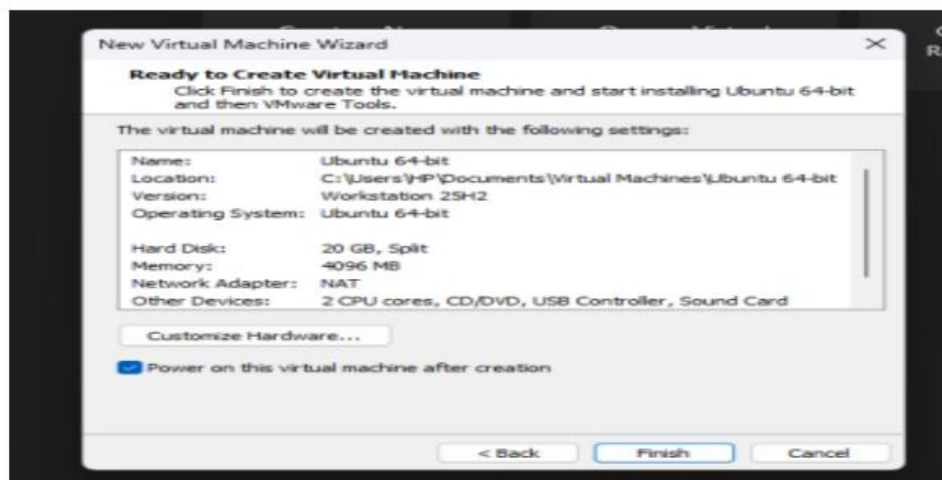
Step 26: Disk Capacity

Set the maximum disk size, such as 20 GB, which is plenty for a basic Ubuntu desktop installation. Select Split virtual disk into multiple files to make it easier to move the VM to different computers or drives later.



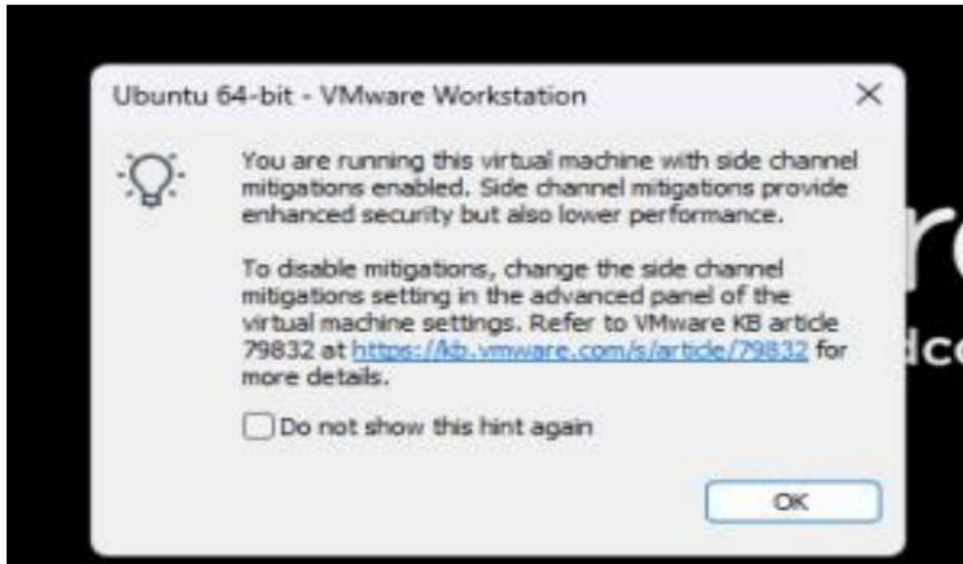
Step 27: Disk File Name

Confirm the file name for the virtual disk file, which typically ends in the .vmdk extension. Click Next to finalize the storage configuration for your new virtual machine.



Step 28: Final Review

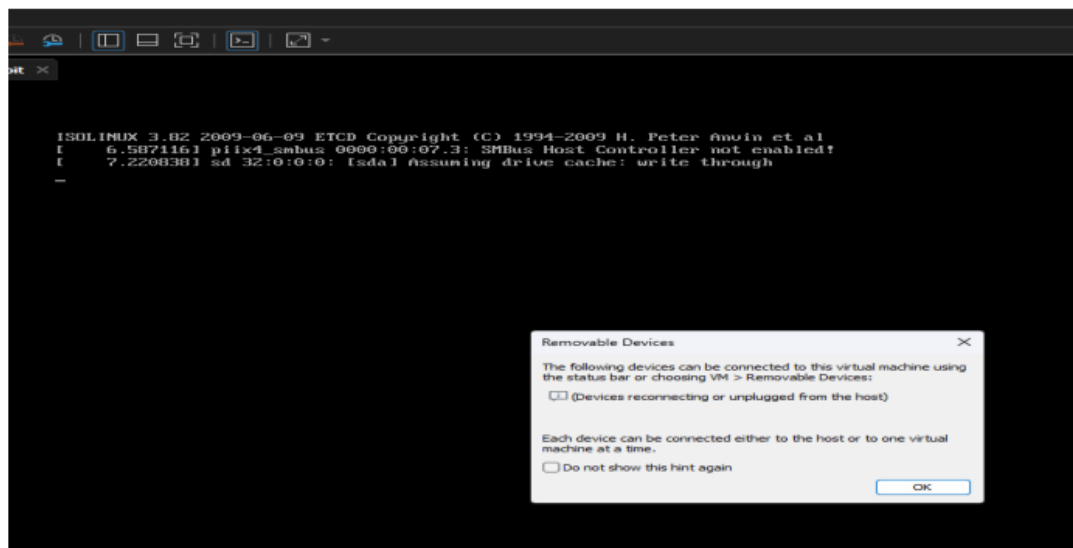
A summary of all hardware settings is displayed; check the box to Power on this virtual machine after creation. Click Finish to exit the wizard and begin the actual OS installation.



Phase 3: Ubuntu Linux Installation Process

Step 29: Side-Channel Mitigations

Upon starting the VM, a notice regarding side-channel mitigations appears, explaining security and performance trade-offs. The user clicks OK to proceed with launching the Ubuntu virtual machine.



Step 30: Initial Boot Sequence

The virtual machine boots into the ISOLINUX environment, displaying logs as the system initializes. This phase confirms that the virtual hardware is correctly interacting with the Ubuntu ISO image.

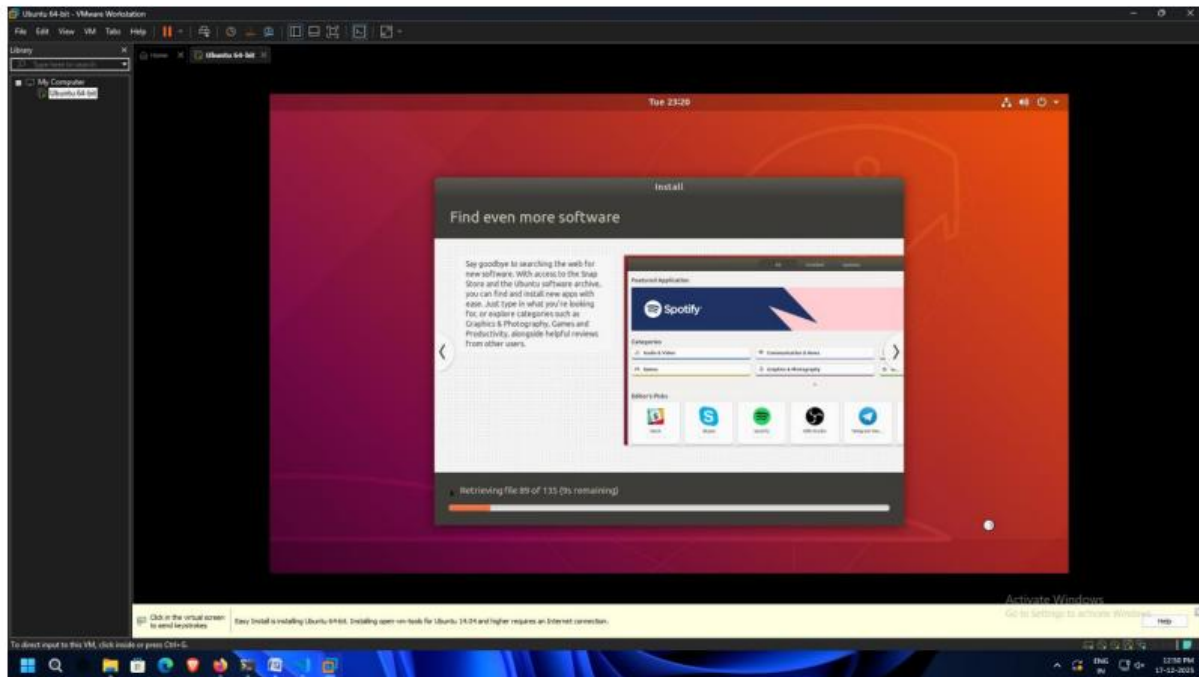


Step 31: Welcome to Ubuntu

The official Ubuntu welcome screen appears as the automated "Easy Install" process begins copying core files. This background process is transferring the Linux kernel and system utilities to the virtual hard drive.

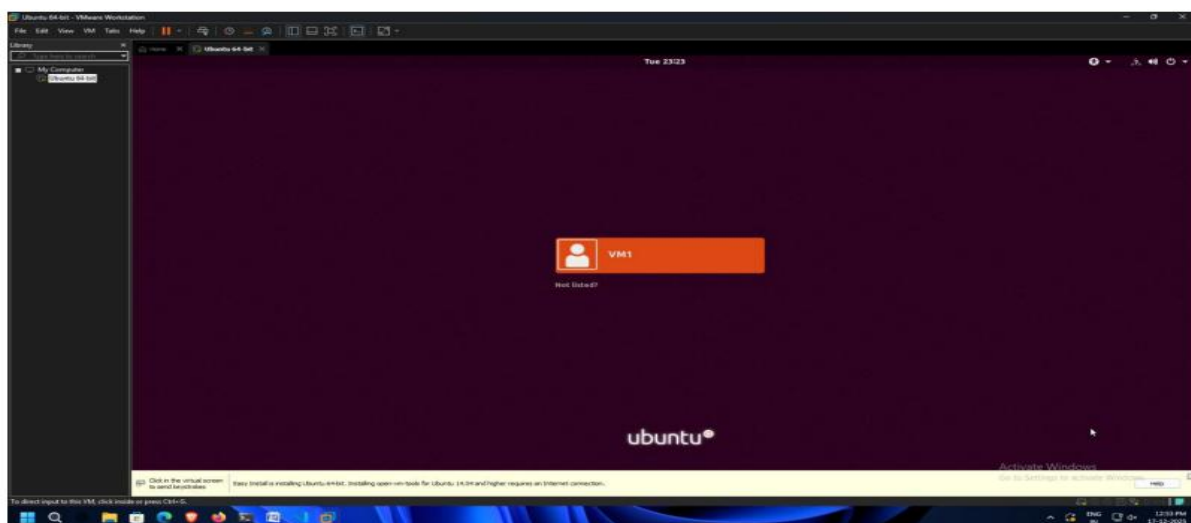
Step 32: Copying Files

The official Ubuntu welcome screen appears while the system begins copying the core files to the disk. This stage is automated, utilizing the personalization settings provided earlier in the setup wizard.



Step 33: Tool Installation

The system automatically begins installing open-vm-tools to ensure a high-performance link between the host and guest. This package enables features like shared folders, clipboard sharing, and automatic screen resizing.



Step 34: Finalizing Installation

The installer finishes copying files and configures system tools like open-vm-tools for better performance. Finally, the user is presented with the login screen for the account "VM1," signifying the successful installation.

Exp 4 — Install C Compiler in VM and Execute Program

Aim

To install a C compiler (GCC) in the virtual machine and execute a simple C program.

Software Requirements

- Oracle VirtualBox
- Ubuntu Linux Virtual Machine
- GCC Compiler
- Terminal

A compiler is a software that converts high-level programming language into machine code. GCC (GNU Compiler Collection) is commonly used in Linux systems to compile C programs. In this experiment, GCC is installed in the virtual machine and used to compile and execute a simple C program.

Procedure

1. Start the Ubuntu virtual machine created in VirtualBox.
2. Open the terminal using Ctrl + Alt + T.
3. Update system packages using the command:
4. `sudo apt update`
5. Install GCC compiler using:
6. `sudo apt install build-essential`
7. Verify installation using:
8. `gcc --version`
9. Create a C program file using:
10. `nano hello.c`
11. Write the C program and save the file.
12. Compile the program using:
13. `gcc hello.c`
14. Execute the program using:
15. `./a.out`
16. Observe the output.

Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello Cloud Lab\n");
```

```
    return 0;
```

```
}
```

Output

Hello Cloud Lab

Exp 5 — Create AWS EC2 Instance and Setup Web Server

Aim

To create an Amazon EC2 instance, configure a web server, associate a public IP address, and allow HTTP (port 80) access using a security group.

Software Requirements

- AWS Account (Free Tier)
- Web Browser
- Internet Connection

Amazon Elastic Compute Cloud (EC2) is a cloud computing service that provides scalable virtual servers on demand. Users can launch virtual machines called instances, configure operating systems, and deploy applications. A security group acts as a firewall to control inbound and outbound traffic. A web server such as Apache is installed to host web pages accessible through the public IP address.

Procedure

1. Create an AWS free tier account and log in to AWS Management Console.
2. Navigate to the EC2 service dashboard.
3. Click on **Launch Instance**.
4. Enter instance name and select **Ubuntu Server (Free Tier)**.
5. Choose instance type **t2.micro**.
6. Create a new key pair and download the .pem file.
7. Configure network settings and allow **HTTP (port 80)** and **SSH (port 22)**.
8. Launch the EC2 instance.
9. Wait until instance state shows **Running**.
10. Connect to the instance using **EC2 Instance Connect**.
11. Install Apache web server using:

```
sudo apt update
```

```
sudo apt install apache2 -y
```

12. Start the Apache server:

```
sudo systemctl start apache2
```

13. Copy the public IP address of the instance.

14. Open a browser and enter:

```
http://<public-ip>
```

15. Observe the Apache default web page.

To create an Amazon EC2 instance, configure a security group to allow HTTP (port 80) access, install and host a web server on the instance, and associate a public (static) IP address with the instance.

Requirements:

- AWS Account
- Internet connection
- Web browser (Chrome / Firefox / Edge)
- Basic knowledge of Linux commands

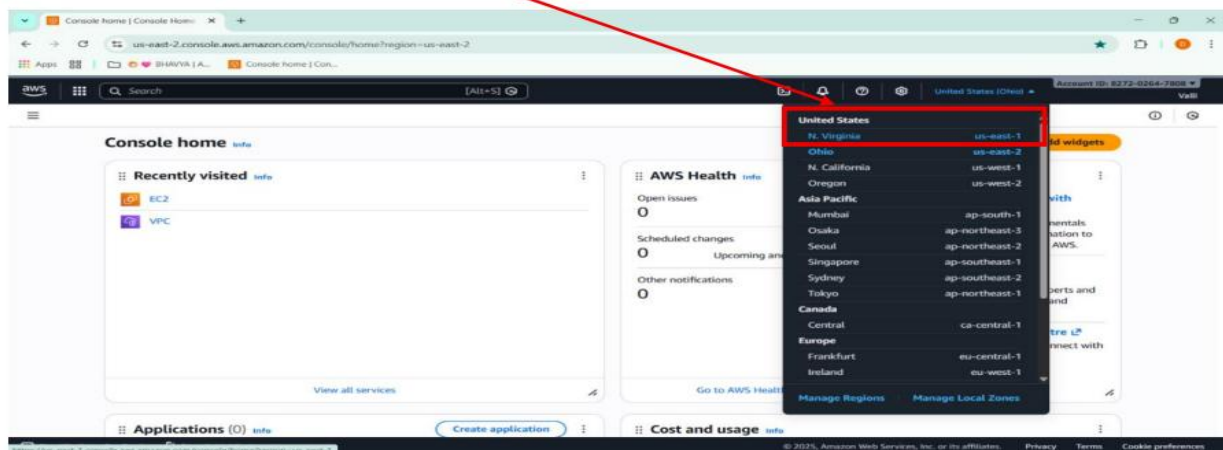
Important Terms:

- **EC2 – Elastic Compute Cloud**
→ A virtual server provided by AWS to run applications.
- **AMI – Amazon Machine Image**
→ A template containing the operating system and required software.
- **HTTP – HyperText Transfer Protocol**
→ Used to transfer web pages over the internet (port 80).
- **SSH – Secure Shell**
→ Used to securely connect to a remote server (port 22).
- **IP Address – Internet Protocol Address**
→ A unique address used to identify a device on a network.
- **Elastic IP**
→ A static public IP address provided by AWS.

Procedure:

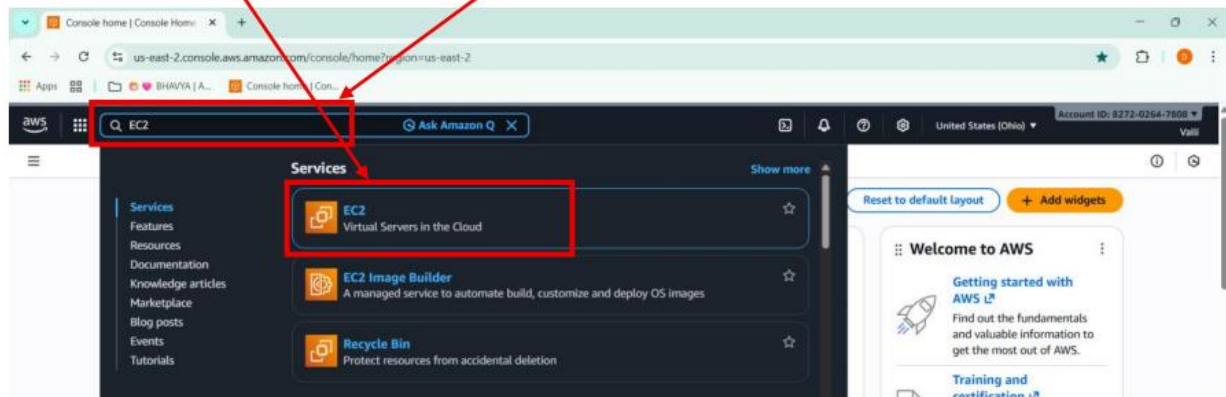
Step 1: Login to AWS Management Console

1. Open a browser and go to <https://aws.amazon.com/>
2. Click **Sign In to the Console** → Login using your AWS credentials
3. Select a region from the top-right corner (Example: US East – N. Virginia / region of your choice)



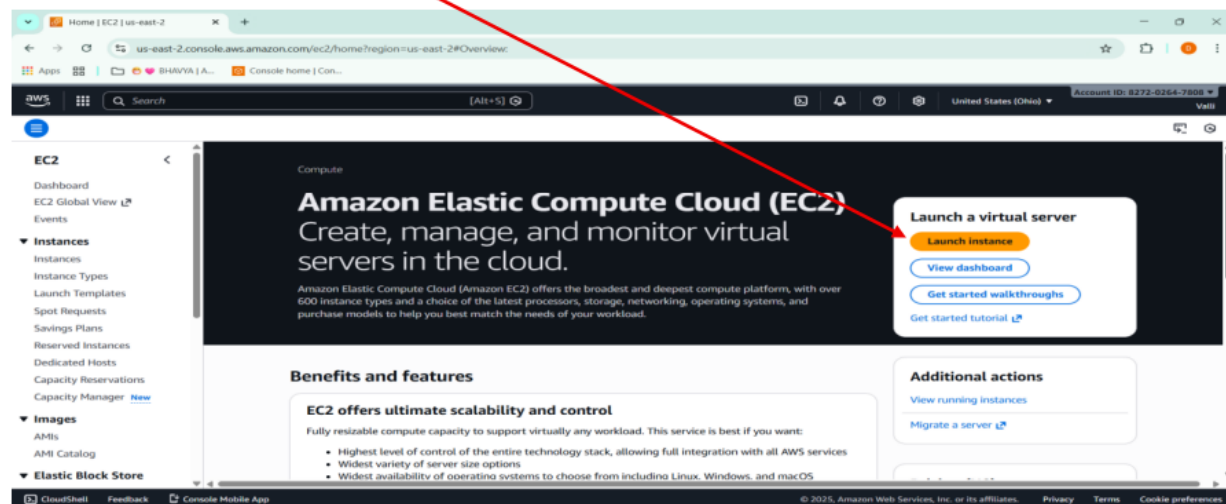
Step 2: Open EC2 Dashboard

1. In the AWS Console search bar, type **EC2**
2. Click **EC2**

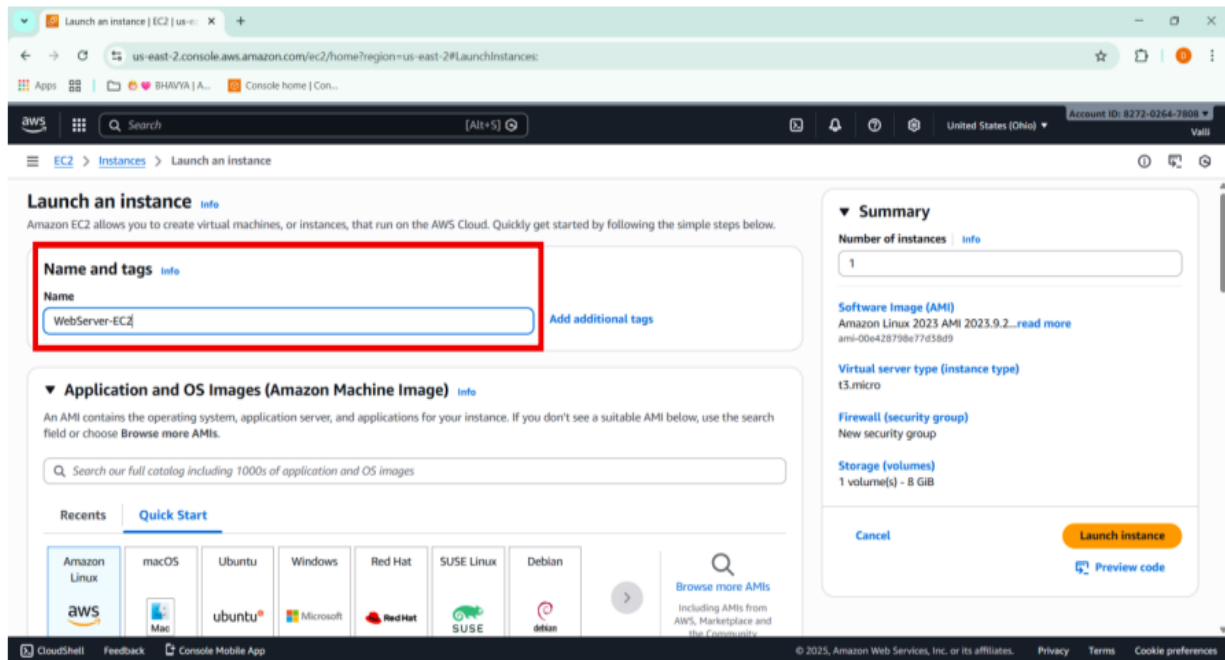


Step 3: Launch a New EC2 Instance

1. Click **Launch Instance**

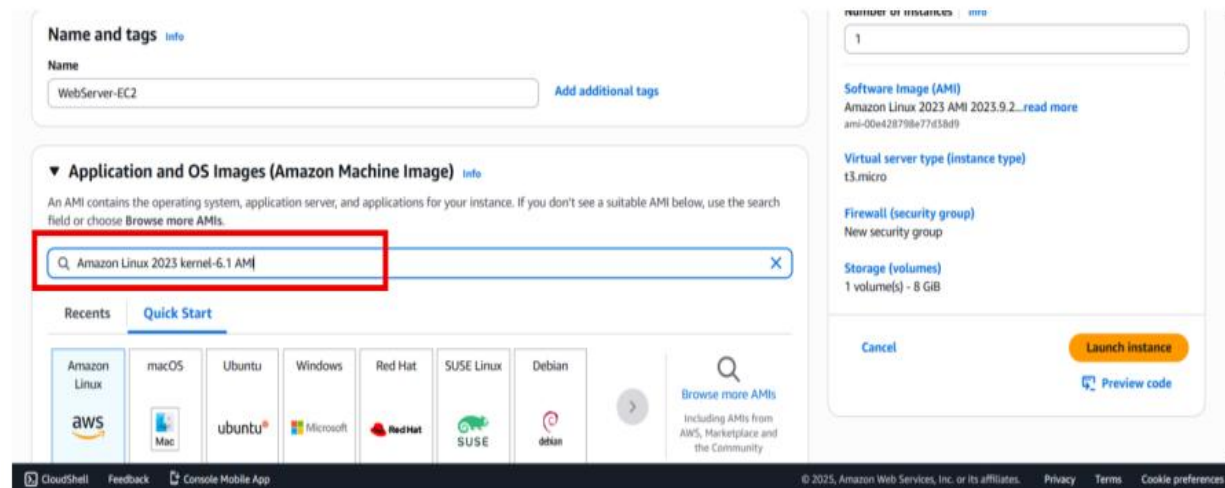


2. Enter **Instance Name**: WebServer-EC2 (Name as per your preference)

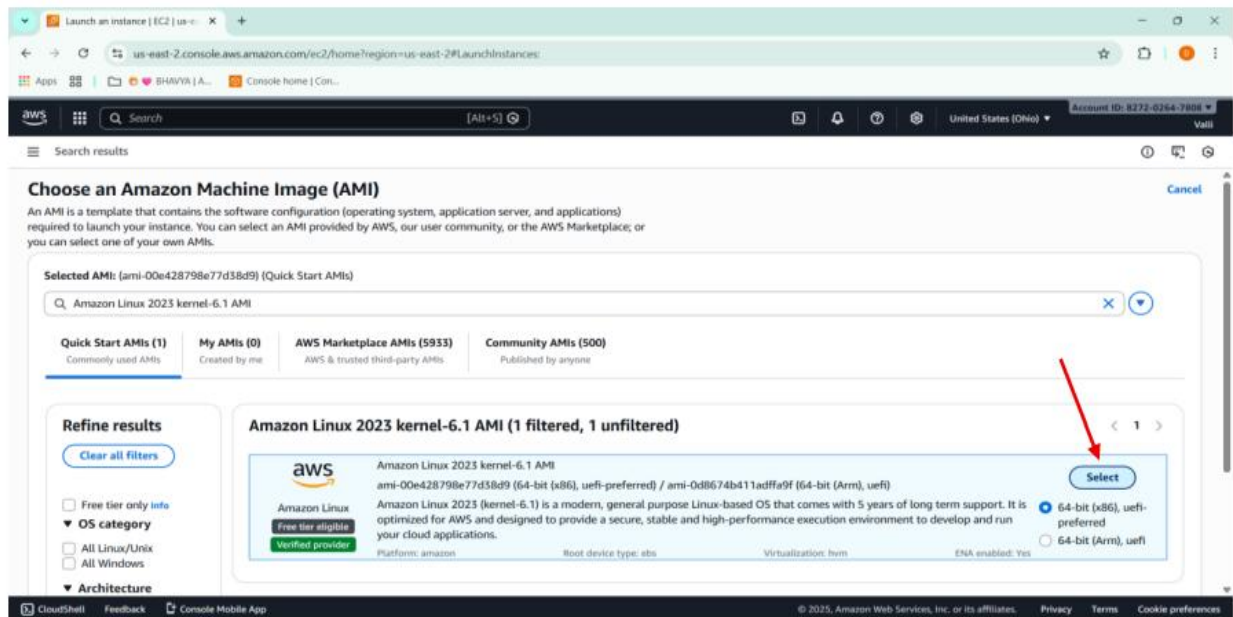


Step 4: Choose Amazon Machine Image (AMI)

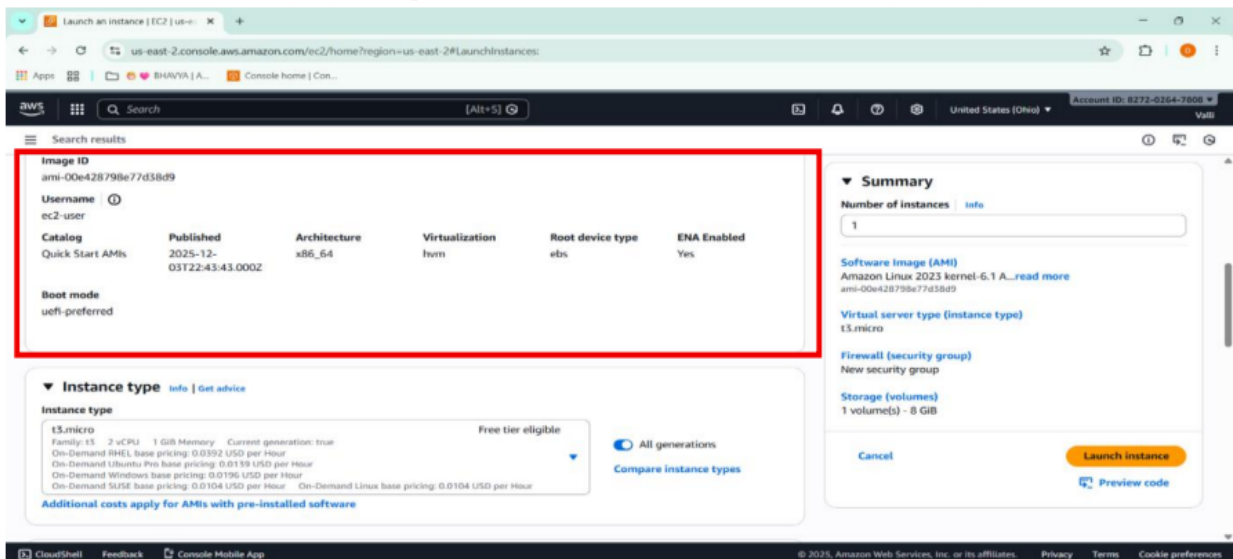
1. Under Quick Start AMIs, select Amazon Linux



2. Choose: Amazon Linux 2023 kernel-6.1 AMI



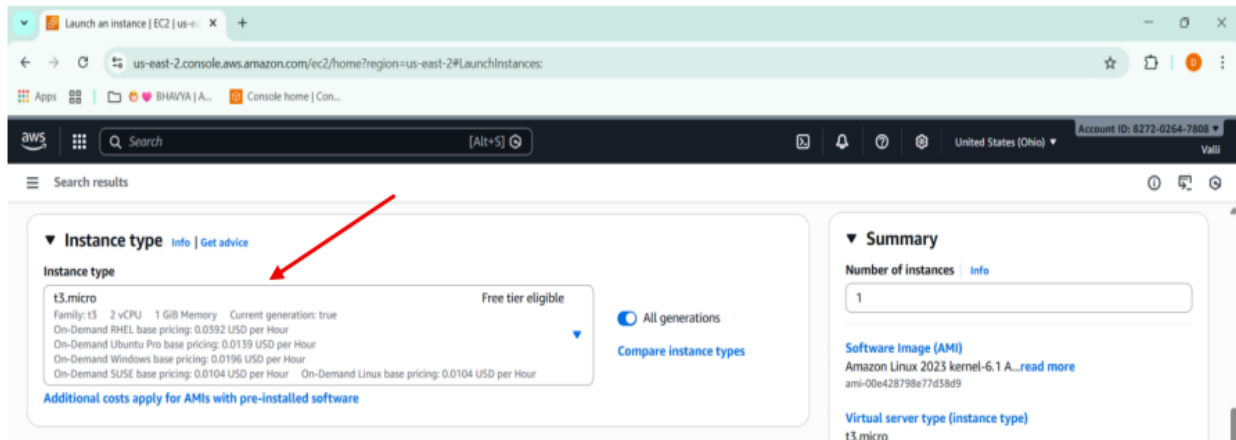
- AMI ID: ami-00e428798e77d38d9
- Architecture: 64-bit (x86), uefi-preferred
- Free Tier Eligible: Yes
- Provider: AWS (Verified)



(This AMI is selected because it is free tier eligible, stable, and suitable for web server experiments.)

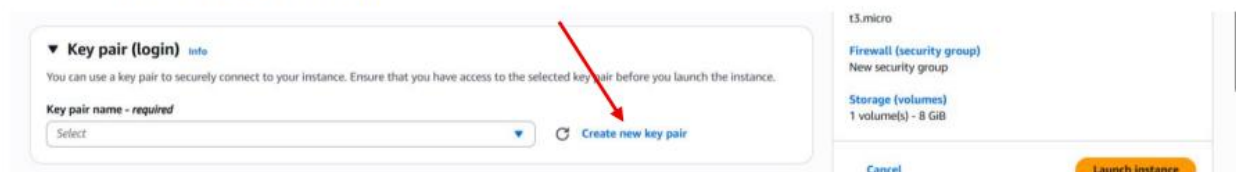
Step 5: Choose Instance Type

1. Select **t2.micro** or **t3.micro** (*Free Tier Eligible*)
2. Click **Next**

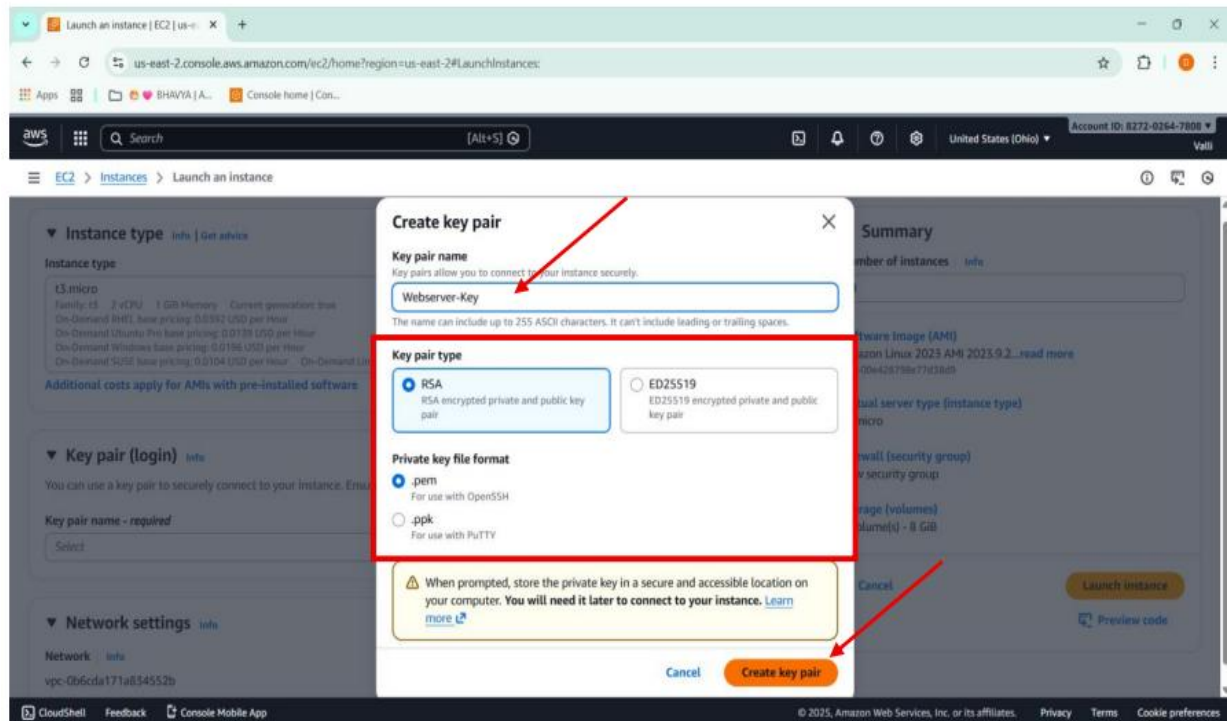


Step 6: Create a Key Pair

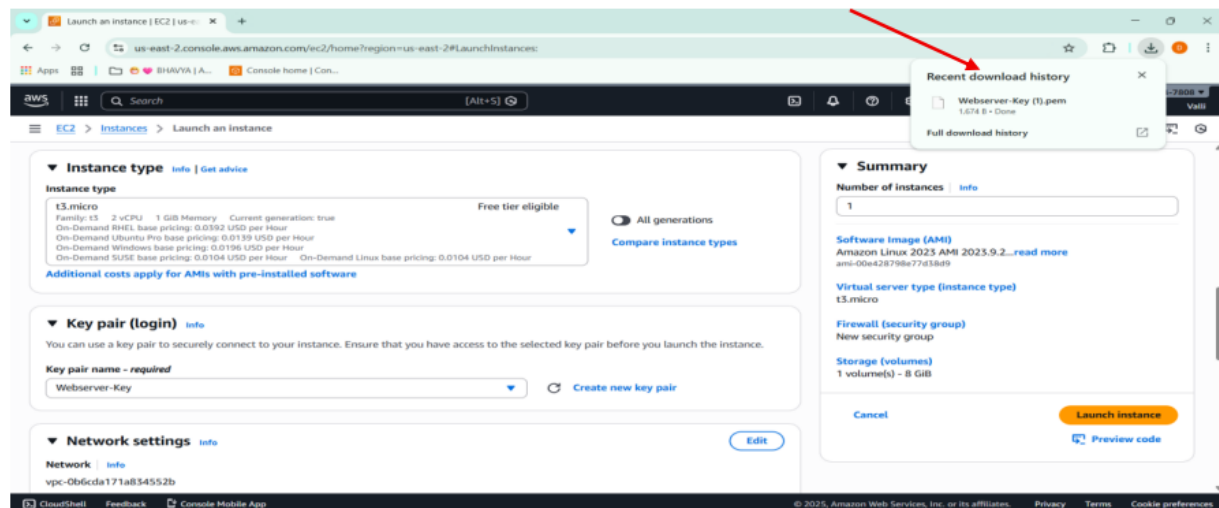
1. Click **Create new key pair**



2. **Key pair name:** Webserver-Key (*Name as per your preference*)
3. **Key pair type:** RSA
4. **Private key format:** .pem
5. Click **Create key pair**



6. The file will download automatically (save it safely)

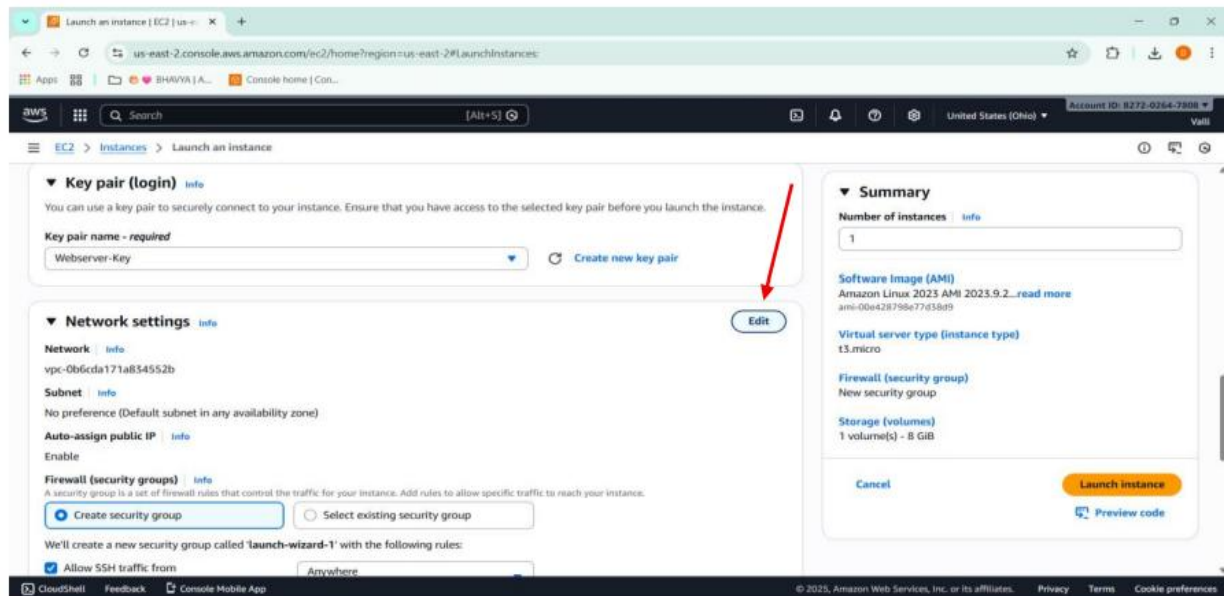


✦ Why we download the key pair?

The key pair is used for secure authentication while connecting to the EC2 instance via SSH. It must be saved safely and not shared, as it acts like a password for the server.

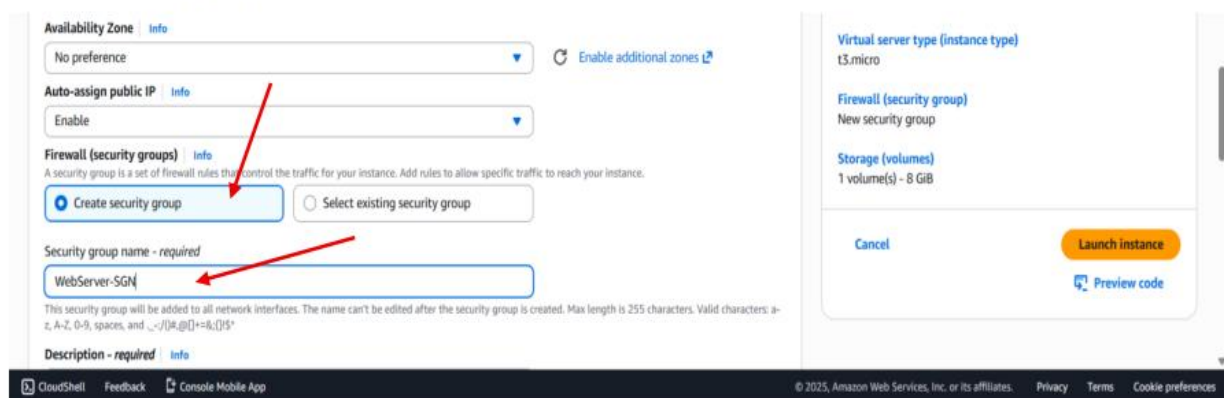
Step 7: Configure Security Group

1. Under Network settings, click Edit



2. Click **Create security group**

3. **Security group name:** WebServer-SGN *(Name as per your preference)*



4. Description: Allow SSH and HTTP access

Inbound Rules:

Type	Protocol	Port	Source
SSH	TCP	22	My IP (0.0.0.0/0 - for connectivity)
HTTP	TCP	80	Anywhere (0.0.0.0/0)

Launch an Instance | EC2 | us-east-2 | console.aws.amazon.com/ec2/home?region=us-east-2#LaunchInstances:

EC2 > Instances > Launch an Instance

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _ (underscore).

Description - required [info](#)
launch-wizard-1 created 2025-12-26T13:11:38.885Z

Inbound Security Group Rules

Security group rule 1 (TCP 22, 223.228.116.141/32) [Remove](#)

Type info	Protocol info	Port range info
ssh	TCP	22
Source type info	Name info	Description - optional info
My IP	EC2: Add CIDR, prefix list or security group	e.g. SSH for admin desktop
Anywhere	223.228.116.141/32	
Custom		
My IP		

Configure storage [info](#) [Advanced](#)

1x 8 GiB gp3 Root volume, 5000 IOPS, Not encrypted

Summary

Number of instances [info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2025.9.2...[read more](#)
ami-00e428799e77e3da9

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch Instance](#) [Preview code](#)

Launch an Instance | EC2 | us-east-2 | console.aws.amazon.com/ec2/home?region=us-east-2#LaunchInstances:

EC2 > Instances > Launch an Instance

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _ (underscore).

Description - required [info](#)
launch-wizard-1 created 2025-12-26T13:11:38.885Z

Inbound Security Group Rules

Security group rule 1 (TCP 22, 223.228.116.141/32) [Remove](#)

Type info	Protocol info	Port range info
ssh	TCP	22
Source type info	Name info	Description - optional info
My IP	EC2: Add CIDR, prefix list or security group	e.g. SSH for admin desktop
Anywhere	223.228.116.141/32	
Custom		
My IP		

[Add security group rule](#)

Configure storage [info](#) [Advanced](#)

1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Summary

Number of instances [info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2025.9.2...[read more](#)
ami-00e428799e77e3da9

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch Instance](#) [Preview code](#)

Launch an Instance | EC2 | us-east-2 | console.aws.amazon.com/ec2/home?region=us-east-2#LaunchInstances:

EC2 > Instances > Launch an Instance

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _ (underscore).

Description - required [info](#)
launch-wizard-1 created 2025-12-26T13:11:38.885Z

Inbound Security Group Rules

Security group rule 1 (TCP 22, 223.228.116.141/32) [Remove](#)

Security group rule 2 (TCP 80, 0.0.0.0/0) [Remove](#)

Type info	Protocol info	Port range info
HTTP	TCP	80
Source type info	Source info	Description - optional info
Anywhere	0.0.0.0/0	e.g. SSH for admin desktop

Configure storage [info](#) [Advanced](#)

1x 8 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Summary

Number of instances [info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2025.9.2...[read more](#)
ami-00e428799e77e3da9

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

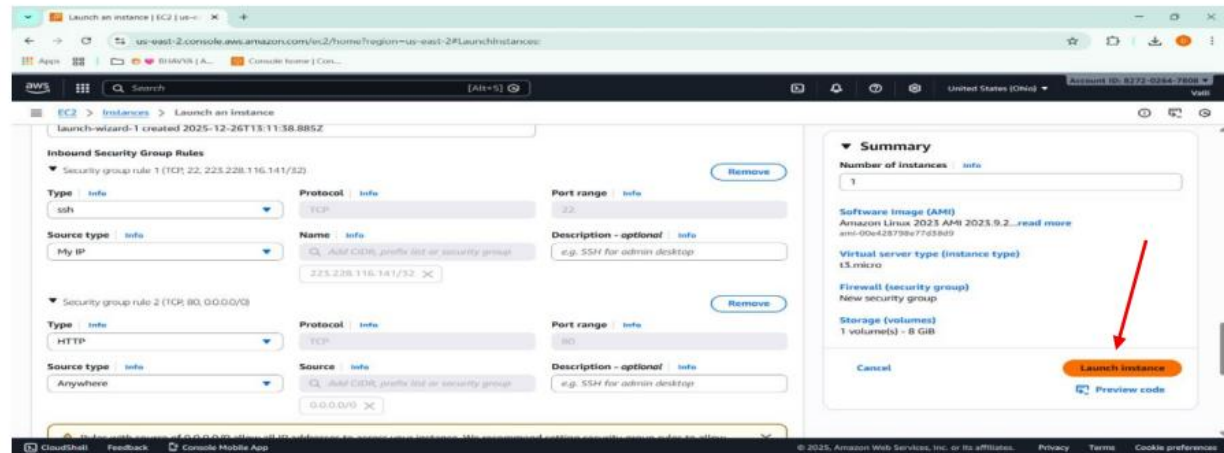
[Cancel](#) [Launch Instance](#) [Preview code](#)

Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

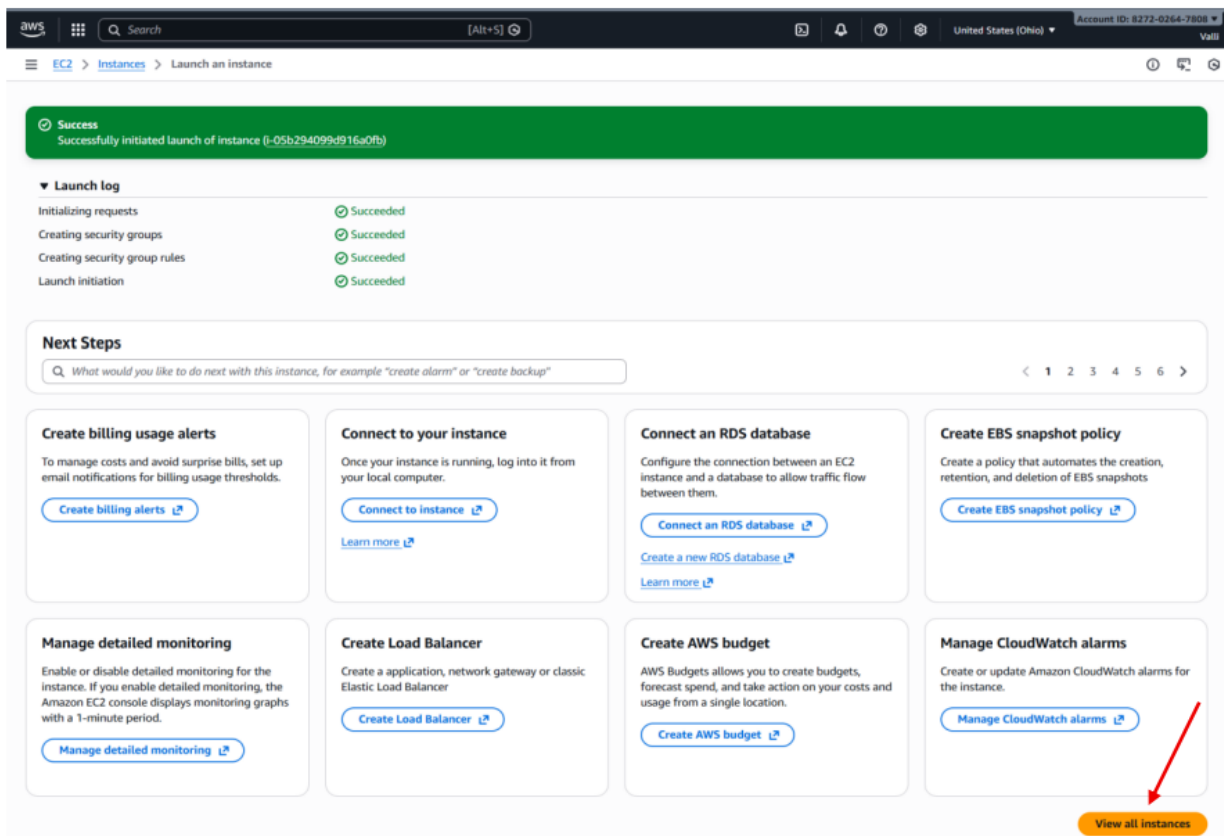
(SSH allows remote login, HTTP allows web access.)

Step 8: Launch the Instance

1. Click Launch Instance



2. Click View Instances

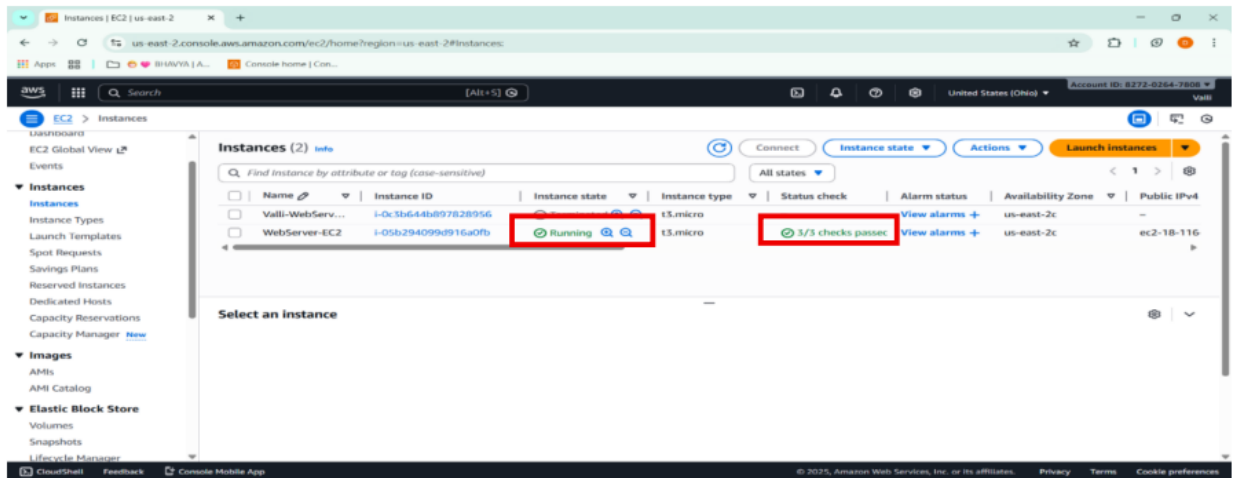


3. Wait until:

- Instance State = **Running**
- Status Checks = **Passed (2/2 or 3/3 depending on AWS console)**

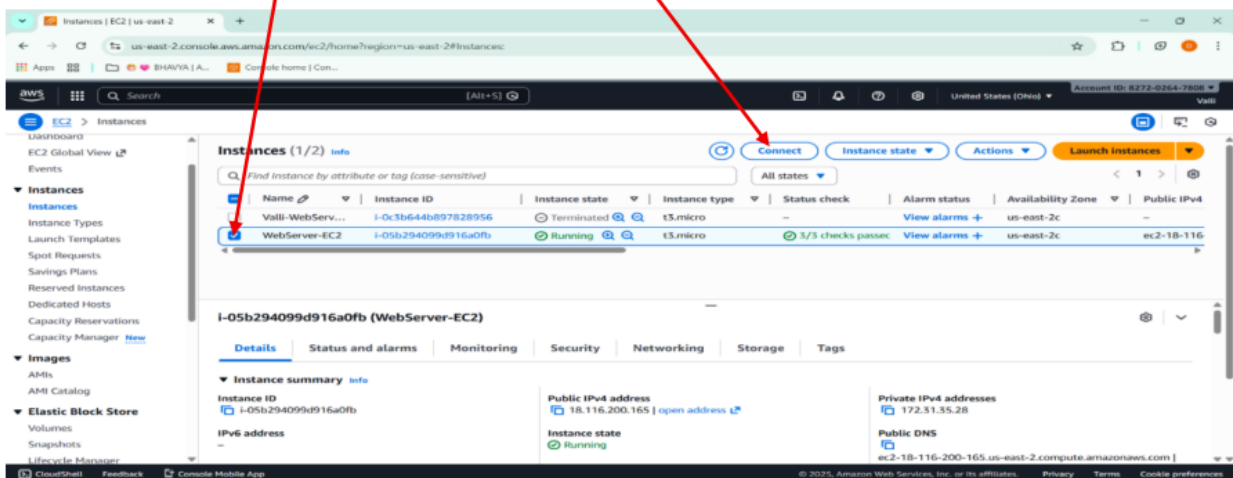
Note: Please be patient while all checks are completed. Do not refresh the page. Only after all status checks pass, the instance is ready for connection.

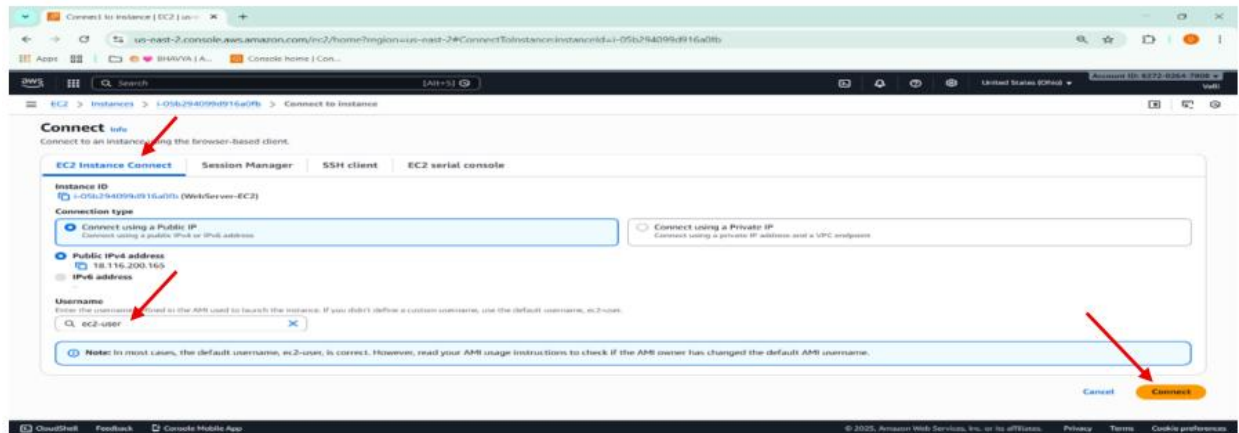
(Only after all status checks pass, the instance is ready for connection.)



Step 9: Connect to the EC2 Instance

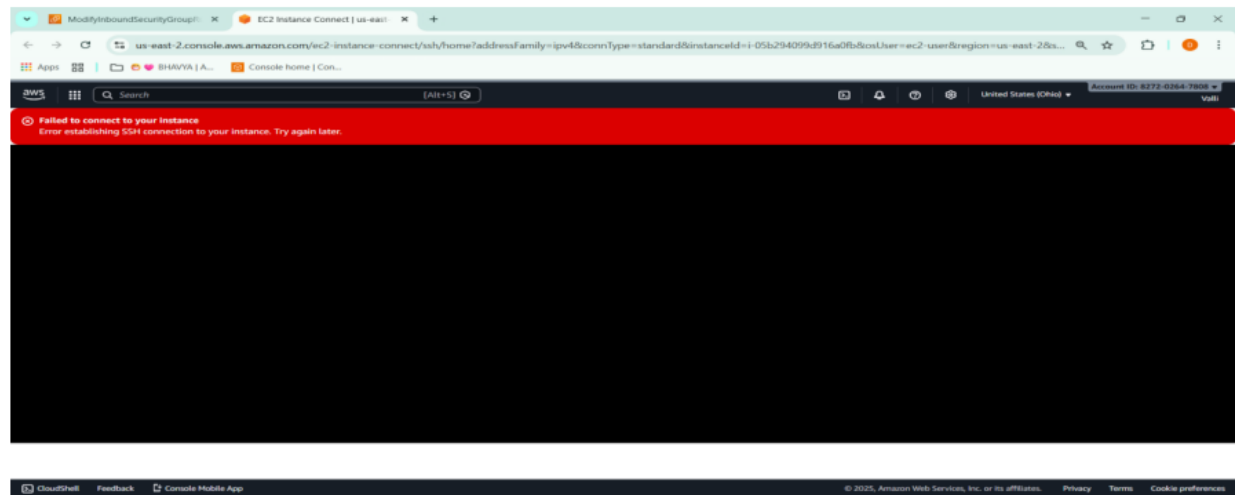
1. Select the instance
2. Click **Connect**





6. You will now see the Linux terminal.

If connection fails, ensure SSH (port 22) is allowed, a public IP is assigned, and status checks are passed.



If Connection Fails

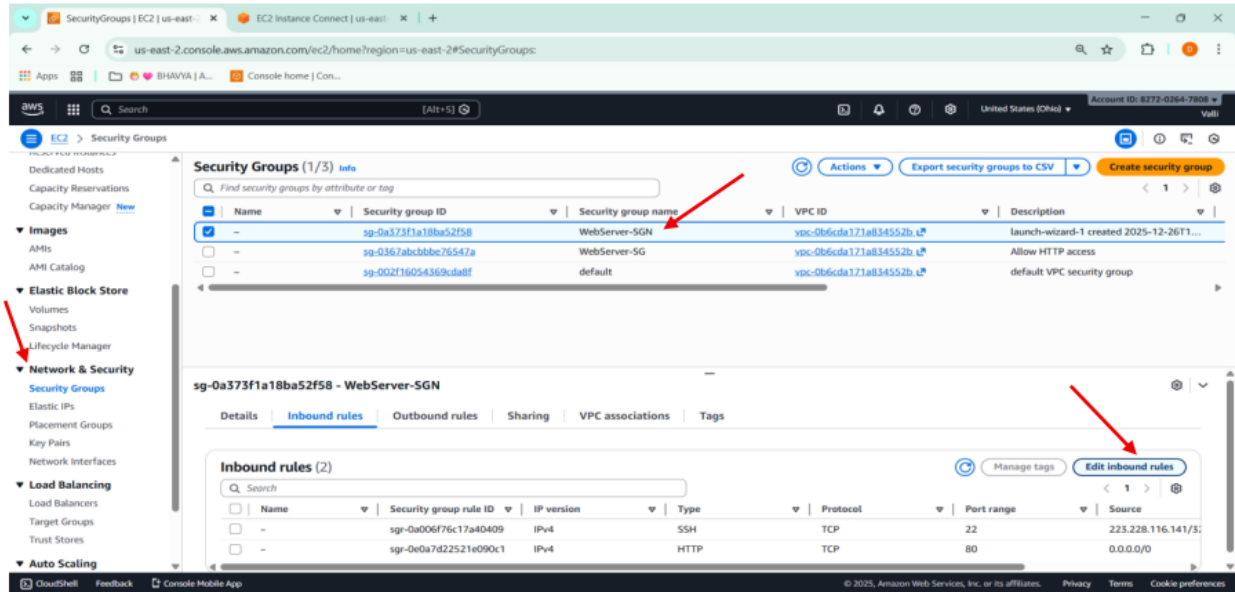
Ensure the following:

- **SSH (port 22)** is allowed in the security group
- The instance has a **public IPv4 address**
- **Status checks** are passed (2/2 or 3/3)

Update Security Group for SSH Access

If SSH access is not working or your IP keeps changing, allow SSH from anywhere:

1. Open the **left sidebar**
2. Go to **Network & Security** → **Security Groups**
3. Select **WebServer-SGN**
4. Click **Edit inbound rules**

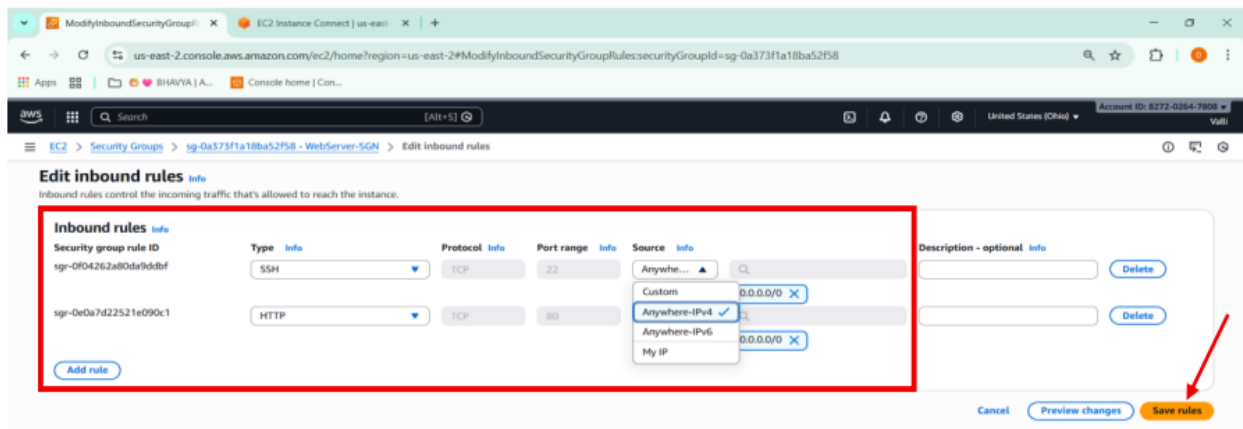


5. Locate SSH (port 22)

6. Change Source:

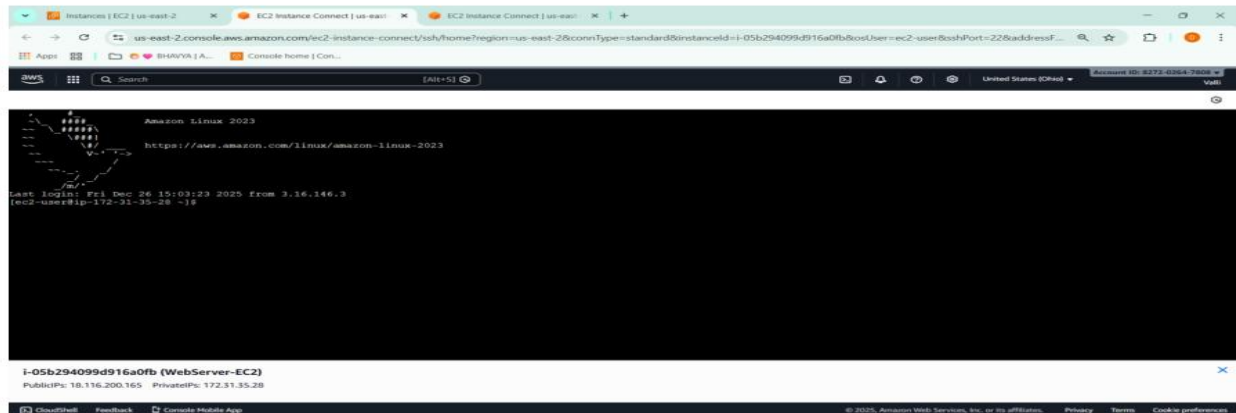
- o Select **Anywhere-IPv4**
- o OR manually enter 0.0.0.0/0

7. Click **Save rules**



Reconnect to the Instance

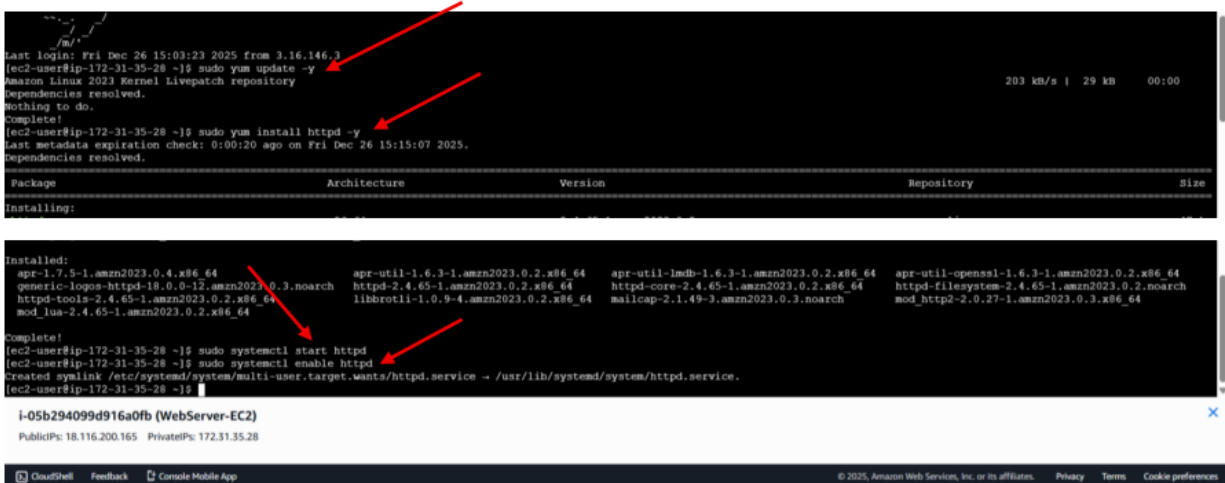
1. Go to **Instances**
2. Select your instance
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**
6. The **Linux terminal** should now open



Step 10: Install Web Server (Apache HTTP Server)

Run the following commands one by one in the terminal:

1. `sudo yum update -y`
2. `sudo yum install httpd -y`
3. `sudo systemctl start httpd`
4. `sudo systemctl enable httpd`



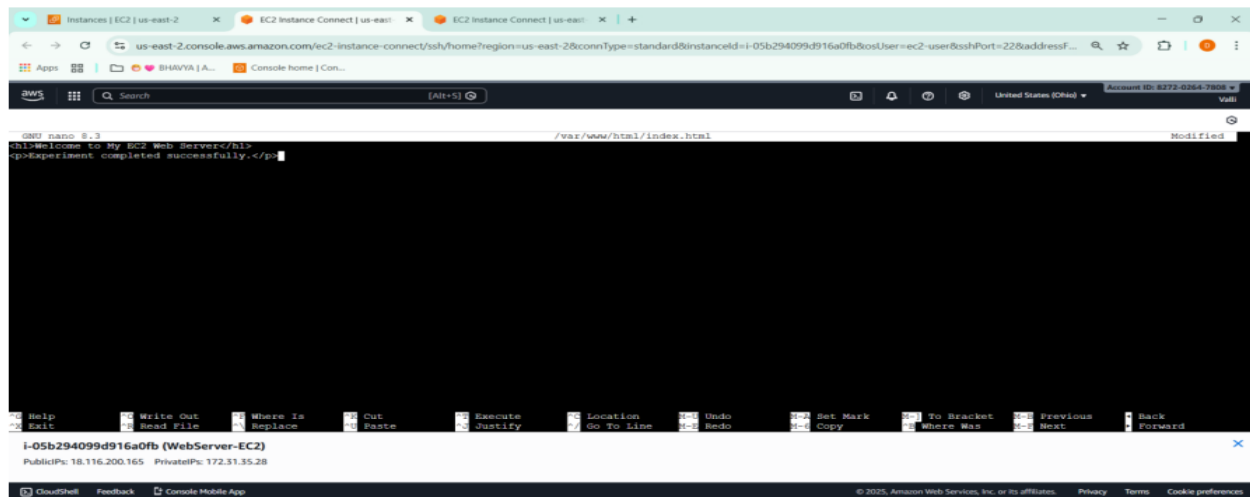
Step 11: Create a Web Page

1. Run the command: `sudo nano /var/www/html/index.html`
2. Add content (*of your choice*):

```
<h1>Welcome to My EC2 Web Server</h1>
```

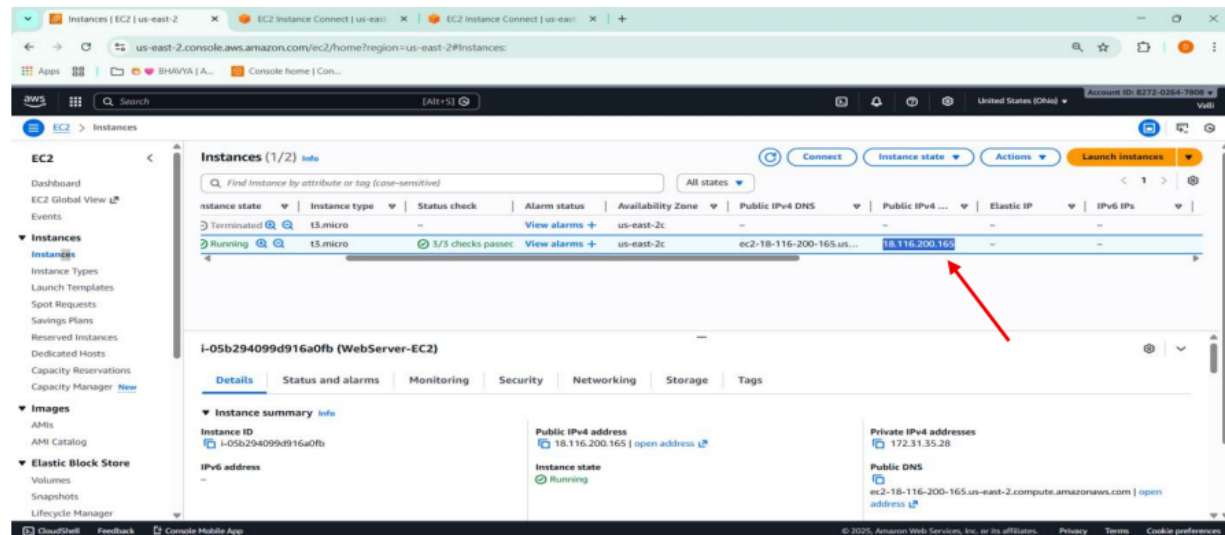
```
<p>Experiment completed successfully.</p>
```

3. Save and exit:
 1. CTRL + O → Enter
 2. CTRL + X

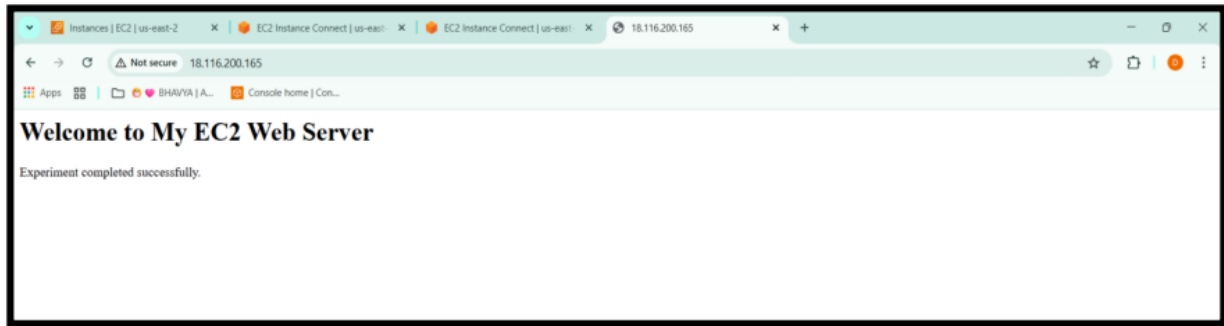
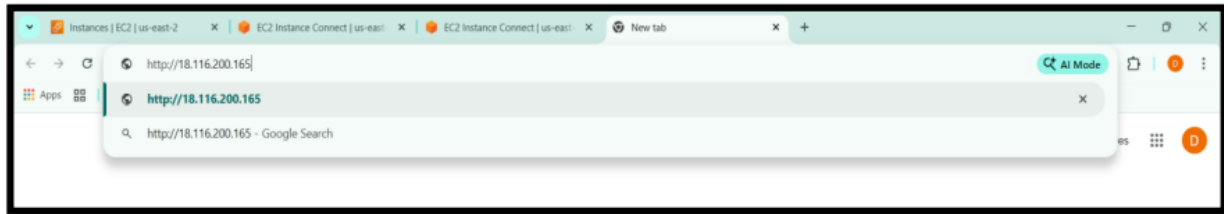


Step 12: Test Web Server Using Public IP

1. Go to EC2 → Instances
2. Copy the Public IPv4 address



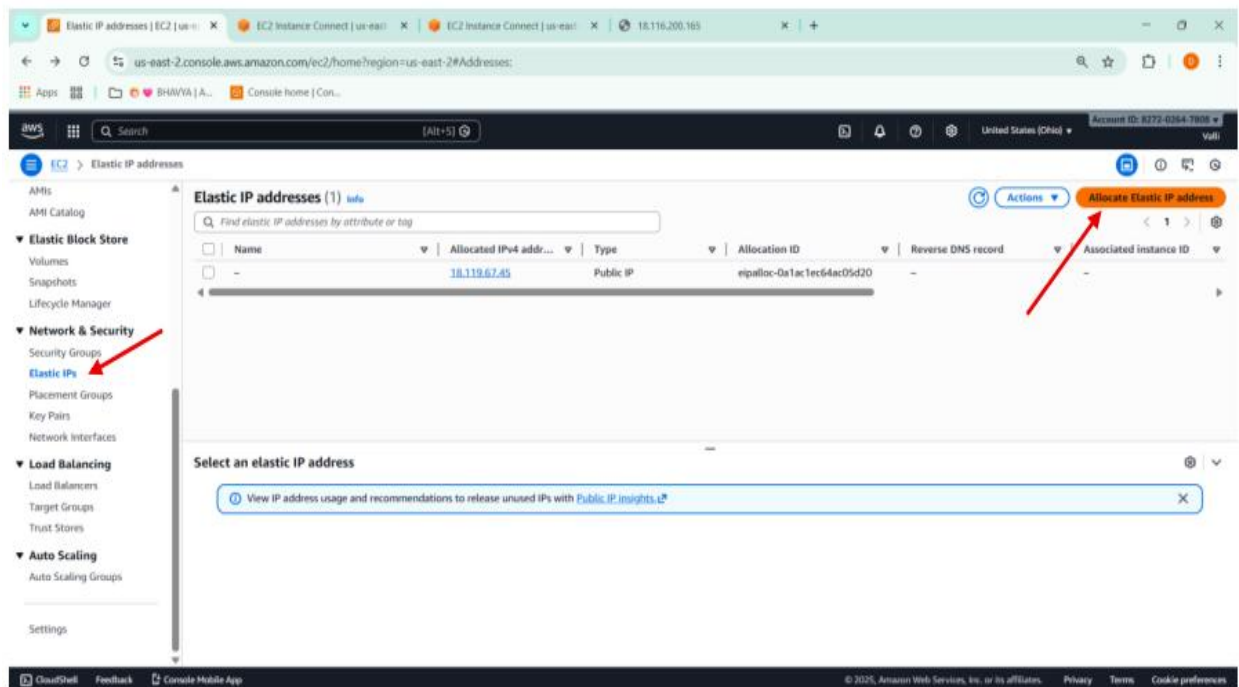
3. Paste in browser:
`http://<Public-IP>` (Example: `http://54.210.123.45`)
✓ Web page should load successfully.



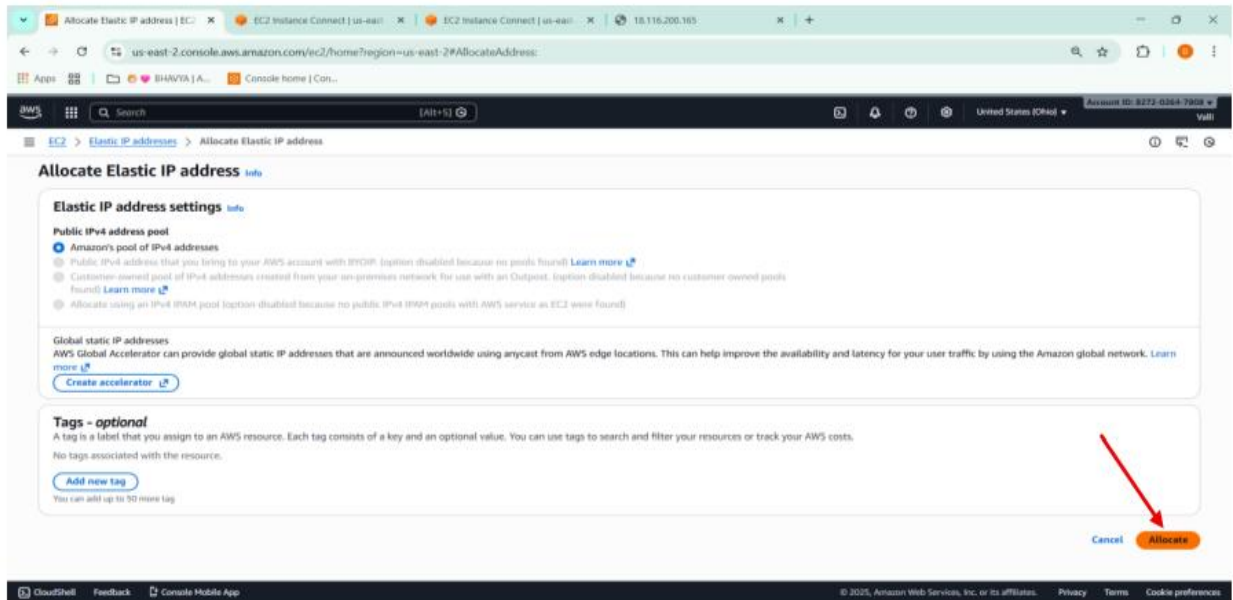
Step 13: Allocate and Associate Elastic IP (Static IP)

Allocate Elastic IP

1. EC2 → Network & Security → Elastic IPs
2. Click Allocate Elastic IP address

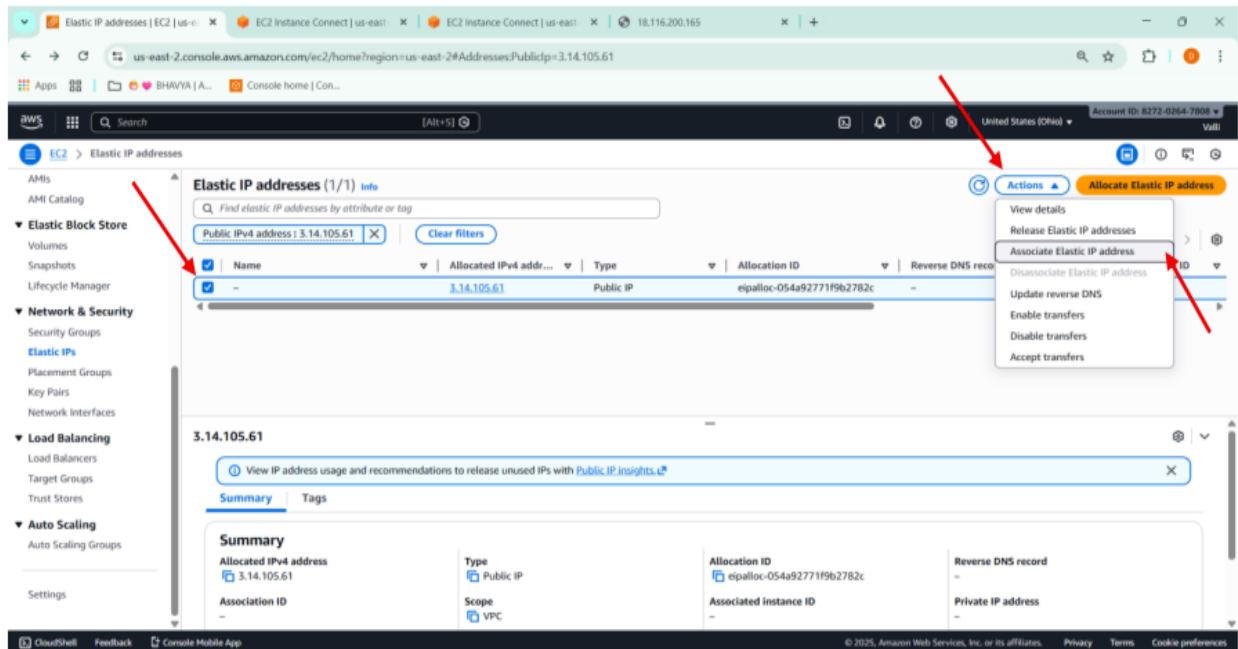


3. Click Allocate



Associate Elastic IP

1. Select the allocated Elastic IP
2. Click Actions → Associate Elastic IP



3. Resource type: **Instance**
4. Select your EC2 instance
5. Click **Associate**

Associate Elastic IP address | EC2 Instance Connect | us-east-2 | 18.116.200.165

us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#AssociateAddress:PublicIp=3.14.105.61

Associate Elastic IP address

Elastic IP address: 3.14.105.61

Resource type
Choose the type of resource with which to associate the Elastic IP address.

☒ Instance
☐ Network interface

If you associate an Elastic IP address with an instance that already has an Elastic IP address associated, the previously associated Elastic IP address will be disassociated, but the address will still be allocated to your account. [Learn more](#)

If no private IP address is specified, the Elastic IP address will be associated with the primary private IP address.

Instance
Choose an instance

i-05b29409d916a0fb (WebServer-EC2) - running

Choose a private IP address

Reassociation
Specify whether the Elastic IP address can be reassociated with a different resource if it already associated with a resource.

☐ Allow this Elastic IP address to be reassociated

Cancel Associate

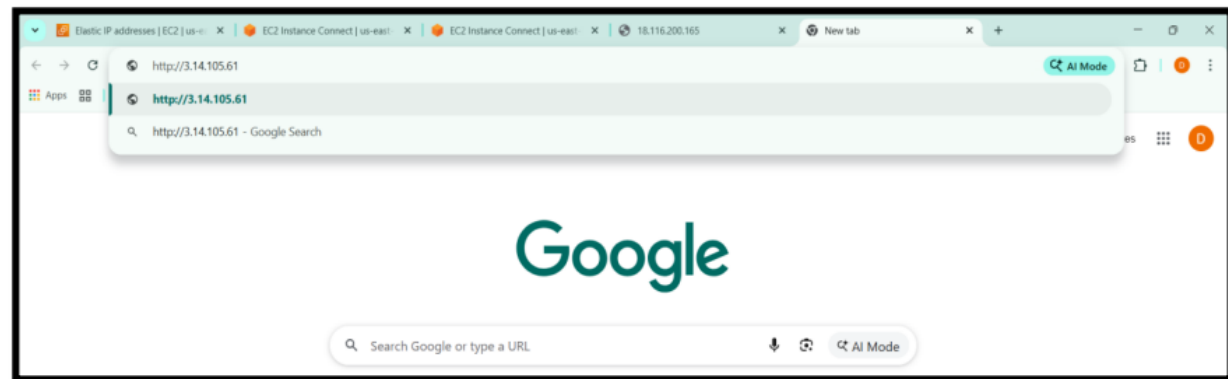
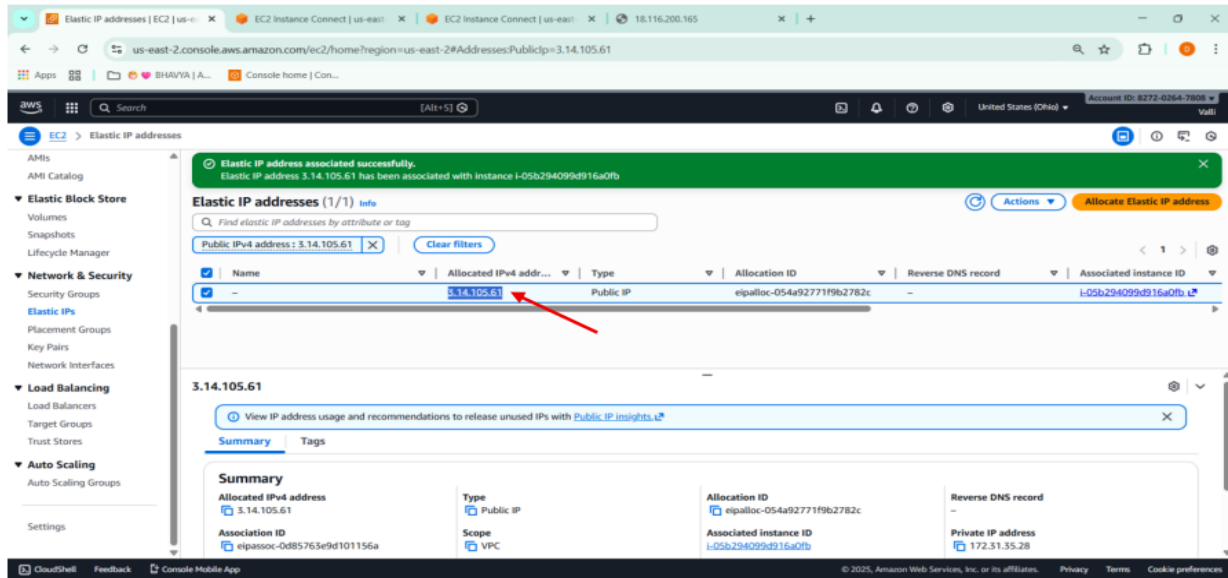
(Elastic IP provides a static public IP that does not change.)

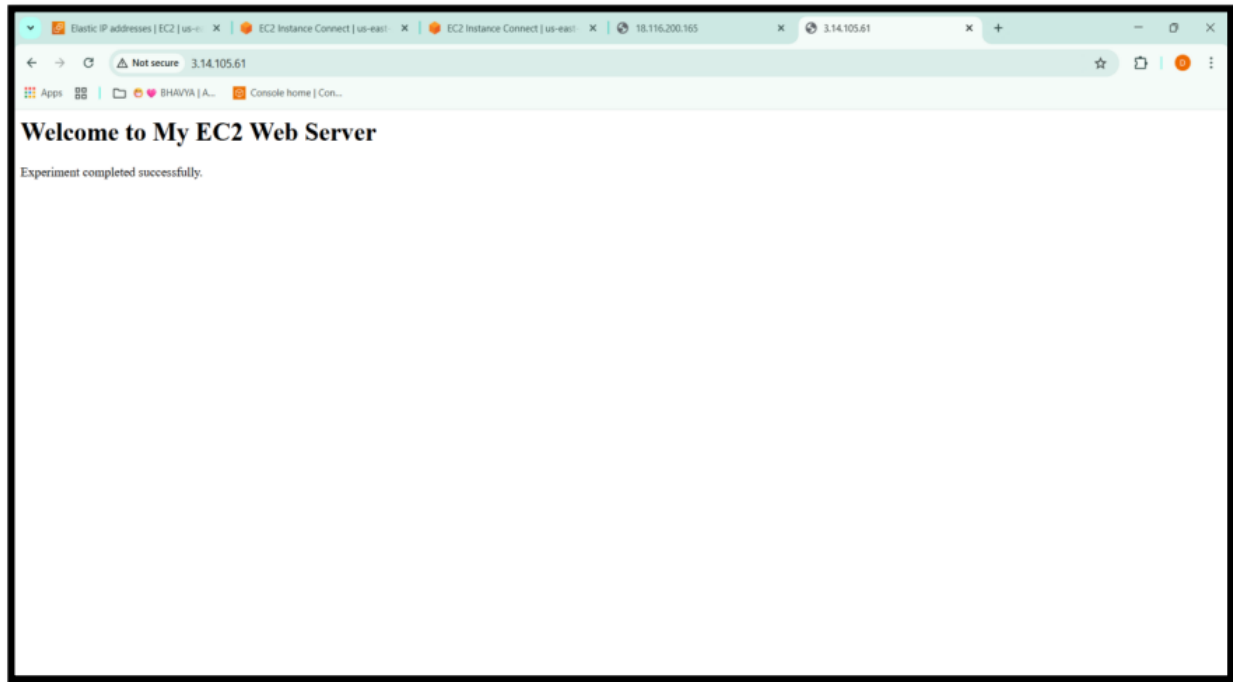
Step 14: Final Testing

1. Copy the **Elastic IP address**
2. Paste in browser:

http://<Elastic-IP> (Example: http://3.88.45.120)

✓ Web server is accessible using static IP.





Result: An Amazon EC2 instance was successfully created using Amazon Linux 2023. Apache web server was installed and configured. A security group was created to allow SSH and HTTP access, and an Elastic IP address was associated. The hosted web page was accessed successfully through the public IP address.

Exp– 7

Aim

To demonstrate the installation of Google App Engine and create a simple Hello World web application using Python.

In this experiment, we install Google Cloud CLI and configure the environment to run a Python-based web application. A simple Flask application is created and executed locally to simulate cloud deployment.

Procedure

Steps to do in Localhost:

1. Download the Google Cloud CLI installer from the official website.
2. Open PowerShell and run the command:

Invoke-WebRequest

```
https://dl.google.com/dl/cloudsdk/channels/rapid/GoogleCloudSDKInstaller.exe -OutFile  
GoogleCloudSDKInstaller.exe
```

3. Run the installer using:

```
.\GoogleCloudSDKInstaller.exe
```

4. Create a new directory:

```
mkdir python-webapp
```

```
cd python-webapp
```

5. Create a Python file named webapp.py.

6. Install Flask module using:

```
pip install flask
```

7. Run the application:

```
python webapp.py
```

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Hello World - Python Web App"
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Output

Open browser - localhost//127.0.0.1:5000



```
Google Cloud SDK Shell x Windows PowerShell x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lab> Invoke-WebRequest https://dl.google.com/dl/cloudsdk/channels/rapid/GoogleCloudSDKInstaller.exe -OutFile GoogleCloudSDKInstaller.exe
PS C:\Users\lab> .\GoogleCloudSDKInstaller.exe
PS C:\Users\lab>
PS C:\Users\lab> mkdir python-app

Directory: C:\Users\lab

Mode                LastWriteTime         Length Name
----                -
d-----         1/7/2026  11:45 AM             python-app

PS C:\Users\lab> cd python-app
PS C:\Users\lab\python-app> notepad app.py
PS C:\Users\lab\python-app> pip install flask
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: flask in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (3.1.2)
Requirement already satisfied: blinker<1.9.0 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from flask) (3.1.4)
Requirement already satisfied: colorama in c:\users\lab\appdata\local\packages\pythonsoftwarefoundation.python.3.13.qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from click>=8.1.3->flask) (0.4.6)
PS C:\Users\lab\python-app> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 478-914-354
127.0.0.1 - - [07/Jan/2026 11:46:40] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Jan/2026 11:46:40] "GET /favicon.ico HTTP/1.1" 404 -
```

n

Exp-8

Aim

Operate a Docker container and set it up as a webserver (e.g. apache2 or Python based Flask micro webframework) on the instance. Map the host directory as a data volume for the container

Description : In this experiment, we run a Docker container and start a web server inside it (Apache or Flask). We also map a folder from the host system to the container using volume mapping, so the files in the host folder can be accessed and updated inside the container. Finally, we open the browser using the mapped port (example: localhost:8080 or localhost:5000) to verify that the web server is working successfully.

Docker installation Steps :

1. First install the Docker, Open Google and search Docker after then install for windows
2. During the installation remember to enable the WSL2 it is important to use the backend
3. After installing the Docker restart your system after then open terminal and run the following command the Docker is installed or not Commands : `docker --version` `docker ps`

OPEN (File Explorer)

1. Create the folder name that folder if you want to identify easily .
2. Create a file flask.py and type the following code

flask.py

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Flask Web Services"
```

```
if __name__ == "__main__":
```

```
    app.run(host="0.0.0.0", port=5000)
```

3. Create the file ' requirements.txt ' file and type the following text .

requirements.txt

```
flask
```

4. Create another file ' Dockerfile ' and type the following code

```
FROM python:3.10-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "flask.py"]
```

Cloud Computing Lab

Page No...

5. Open Terminal in VScode ctrl + ~ to open the Terminal and Run the following command to create and Build Docker image

Note : before run the below command you have to open the docker in desktop when ever you open the docker the server will automatically run and connect then the image will build when you run the below command

“ docker build -t flask-image . “

6.After executing the command they start exporting the layers to build the image in the docker container you can check in docker dashboard then go to containers.

7. After then run the another command as below “ docker run -d -p 5000:5000 --name flask-container flask-image “

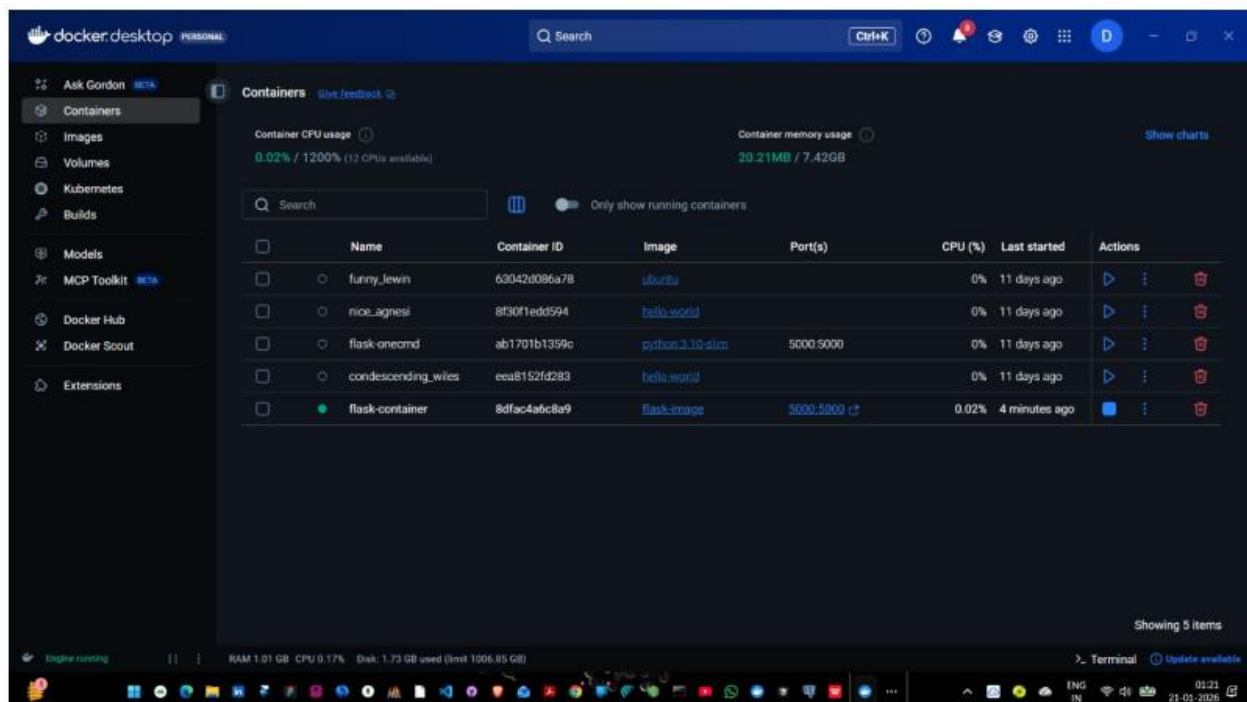
8. The server will run in http://localhost:5000 and if you wan to stop the server “ docker stop flask-container “

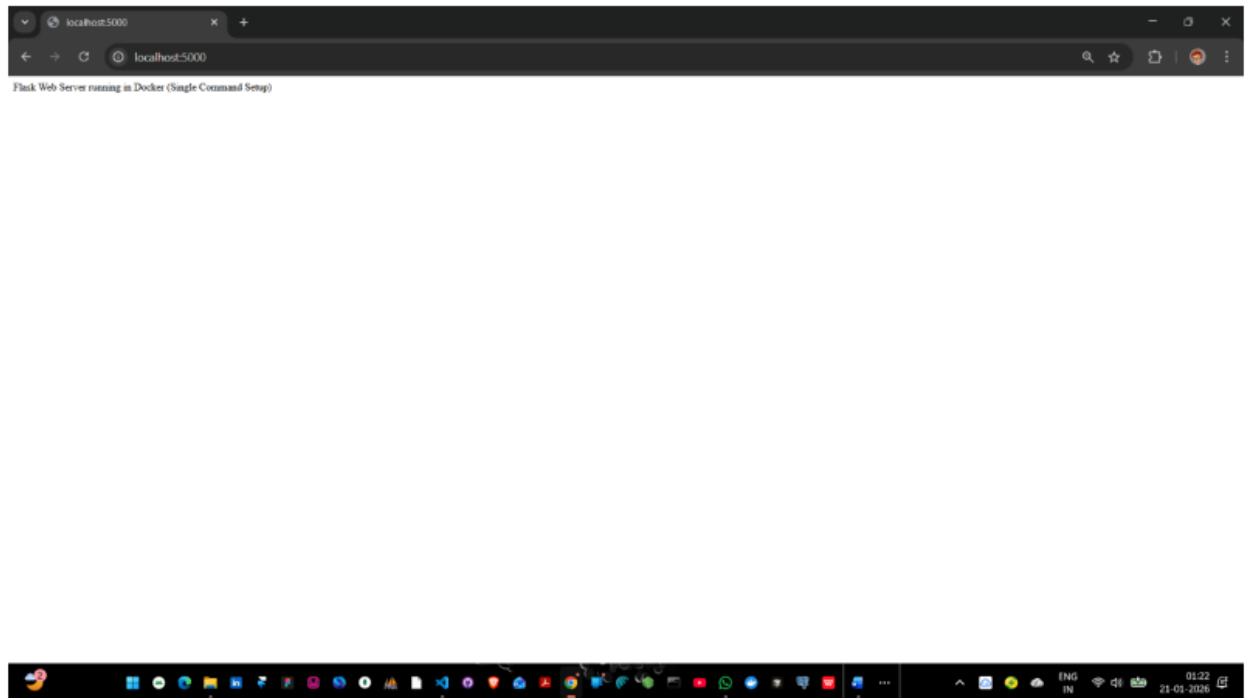
9. To remove or delete the container run the command “ docker ps -a “

10. Then it will display the list of containers what you build look up the last coloum containers names run the command to delete the particular of the container use the ‘ NAMES ‘

11.For example assume that the container is ‘flask-container’ then run the command to remove “ docker rm -f flask-container “

Output





Exp – 9

Aim

To find a procedure to transfer files from one virtual machine to another virtual machine and from one container to another container.

File transfer is an important operation in cloud environments. Virtual machines and containers often need to exchange data. This can be achieved using network-based tools like SSH, SCP, or Docker commands. These methods ensure secure and efficient data transfer between systems.

Procedure

A. Transfer Files Between Virtual Machines

Method 1: Using SCP (Secure Copy)

1. Ensure both virtual machines are connected to the same network.
2. Install SSH in both VMs (if not installed):

```
sudo apt install openssh-server
```

3. Find IP address of destination VM:

```
ip addr
```

4. From source VM, transfer file using:

```
scp filename user@destination_ip:/home/user/
```

5. Enter password of destination VM.
6. File will be copied successfully.

Method 2: Using Shared Folder (VirtualBox)

1. Open VirtualBox → Settings → Shared Folders.
2. Add a folder from host system.
3. Access the folder in both VMs.
4. Copy files between VMs using shared folder.

B. Transfer Files Between Containers

Method 1: Using Docker Copy Command

1. Check running containers:

```
docker ps
```

2. Copy file from host to container:

```
docker cp file.txt container1:/app/
```

3. Copy file from one container to another:

```
docker cp container1:/app/file.txt container2:/app/
```

Method 2: Using Volume Mapping

1. Create a shared folder in host system.

2. Run containers with volume mapping:

```
docker run -v host_folder:/app container_name
```

3. Both containers access same folder.

Output

- File successfully transferred between virtual machines.
- File successfully shared between Docker containers.

Exp – 10

Aim

To launch a virtual machine using TryStack (Online OpenStack Demo Platform).

OpenStack is an open-source cloud platform used to create and manage virtual machines. TryStack provides a free online environment to practice OpenStack without installing any software locally. It allows users to launch instances (virtual machines) using the Horizon dashboard.

Procedure

1. Open the TryStack website in a browser.
2. Register for a TryStack account.
3. Login to the OpenStack Horizon dashboard.
4. Navigate to:
 Compute → Instances
5. Click on Launch Instance.
6. Enter instance name and details.
7. Select an image (Ubuntu Linux).
8. Choose a flavor (RAM, CPU, storage).
9. Configure network settings.
10. Click Launch.
11. Wait until instance status becomes Running.
12. Click Console to access the virtual machine.

Output

The virtual machine is launched successfully and the console displays the operating system interface.

Exp – 11

Aim

To install Hadoop in a single node cluster and execute a simple MapReduce application (WordCount).

Hadoop is an open-source framework used for distributed storage and processing of large datasets. It uses HDFS (Hadoop Distributed File System) for storage and MapReduce for processing. In a single node cluster, Hadoop runs on one machine for learning and testing purposes.

Procedure

Step 1: Check Java Installation

Check if Java is installed:

```
java -version
```

If Java is not installed, run:

```
sudo apt update
```

```
sudo apt install openjdk-8-jdk -y
```

Verify Java installation again:

```
java -version
```

Step 2: Download Hadoop

Download Hadoop from the Apache repository:

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

Step 3: Extract Hadoop

Extract the Hadoop tar file:

```
tar -xvzf hadoop-3.3.6.tar.gz
```

Rename the extracted folder:

```
mv hadoop-3.3.6 hadoop
```

Step 4: Verify Hadoop Folder

Check if the Hadoop folder exists:

```
ls
```

Navigate into the Hadoop directory:

```
cd hadoop
```

Verify Hadoop files:

```
ls
```

Return to the home directory:

```
cd
```

Step 5: Set JAVA_HOME Environment Variable

1. Find Java Installation Path

Run the following command to find where Java is installed:

```
readlink -f $(which java)
```

The output should be something like /usr/lib/jvm/java-8-openjdk-amd64/bin/java.

2. Set JAVA_HOME in .bashrc

Open .bashrc file:

```
nano ~/.bashrc
```

Add the following lines at the bottom (replace the Java path with the one you found above):

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

```
export PATH=$PATH:$JAVA_HOME/bin
```

Save and exit (Press CTRL + X, then Y, then Enter).

3. Apply the changes:

```
source ~/.bashrc
```

Step 6: Configure Hadoop Environment Variables

Open .bashrc file again:

```
nano ~/.bashrc
```

Add the following Hadoop environment variables at the bottom:

```
export HADOOP_HOME=/home/levi/hadoop
```

```
export HADOOP_INSTALL=$HADOOP_HOME
```

```
export HADOOP_MAPRED_HOME=$HADOOP_HOME
```

```
export HADOOP_COMMON_HOME=$HADOOP_HOME
```

```
export HADOOP_HDFS_HOME=$HADOOP_HOME
```

```
export YARN_HOME=$HADOOP_HOME
```

```
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
```

```
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
```

Save and exit (Press CTRL + X, then Y, then Enter).

Apply the changes:

```
source ~/.bashrc
```

Step 7: Verify Hadoop Installation

Check the Hadoop version:

```
hadoop version
```

Step 8: Navigate to Hadoop Configuration Folder

Navigate to the Hadoop configuration directory:

```
cd $HADOOP_HOME/etc/hadoop
```

Verify configuration files:

```
ls
```

Step 9: Configure core-site.xml

Open the core-site.xml file:

```
nano core-site.xml
```

Add the following configuration inside <configuration>:

```
<configuration>
```

```
  <property>
```

```
    <name>fs.defaultFS</name>
```

```
    <value>hdfs://localhost:9000</value>
```



```
</property>
</configuration>
```

Save and exit.

Step 10: Configure hdfs-site.xml

Open the hdfs-site.xml file:

```
nano hdfs-site.xml
```

Add the following configuration:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

Save and exit.

Step 11: Create and Configure mapred-site.xml

If mapred-site.xml.template is missing, manually create the file:

```
nano mapred-site.xml
```

Add the following configuration:

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Save and exit.

Step 12: Configure yarn-site.xml

Open the yarn-site.xml file:

```
nano yarn-site.xml
```

Add the following configuration:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Save and exit.

Step 13: Format Hadoop NameNode

Format the Hadoop NameNode:

```
hdfs namenode -format
```

Step 14: Start Hadoop Services

Start HDFS:

```
start-dfs.sh
```

Start YARN:

start-yarn.sh

Step 15: Verify Running Services

Check Hadoop daemons:

jps

Expected output:

NameNode

DataNode

SecondaryNameNode

ResourceManager

NodeManager

Step 16: Create Input Directory in HDFS

Create an input directory in HDFS:

hdfs dfs -mkdir /input

Step 17: Create Sample Input File

Create a sample text file:

nano sample.txt

Example content:

hello hadoop

hello big data

hadoop mapreduce

Step 18: Upload File to HDFS

Upload the sample file to HDFS:

hdfs dfs -put sample.txt /input

Verify the file in HDFS:

hdfs dfs -ls /input

Step 19: Run WordCount Example

Run the WordCount example:

hadoop jar \$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar
wordcount /input /output

Step 20: View Output Directory

View the output directory in HDFS:

hdfs dfs -ls /output

Step 21: Display WordCount Result

Display the result of WordCount:

hdfs dfs -cat /output/part-r-00000

Output

Cloud	1
Hadoop	1
hello	2

Exp – 12

Aim

To utilize OpenFaaS (Serverless computing framework) and demonstrate basic event-driven function invocation.

OpenFaaS is a serverless computing platform that allows users to deploy functions without managing servers. It uses Docker containers to run functions and enables event-driven execution. This helps in building scalable and efficient cloud applications.

Procedure

Open terminal in ubuntu and type following commands

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
sudo apt install docker.io -y
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
sudo usermod -aG docker $USER newgrp docker
```

```
nikhil1@nikhil1-VirtualBox:~/temp$ newgrp docker
```

```
docker run hello-world
```

```
nikhil1@nikhil1-VirtualBox:~/temp$ docker run hello-world
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
To generate this message, Docker took the following steps:
```

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

```
To try something more ambitious, you can run an Ubuntu container with:
```

```
$ docker run -it ubuntu bash
```

```
Share images, automate workflows, and more with a free Docker ID:
```

```
https://hub.docker.com/
```

```
For more examples and ideas, visit:
```

```
https://docs.docker.com/get-started/
```

sudo apt install

git -y git clone <https://github.com/openfaas/faasd> --depth=1

cd faasd

./hack/install.sh

docker login

login with user name and password if don't have account then create one in official site

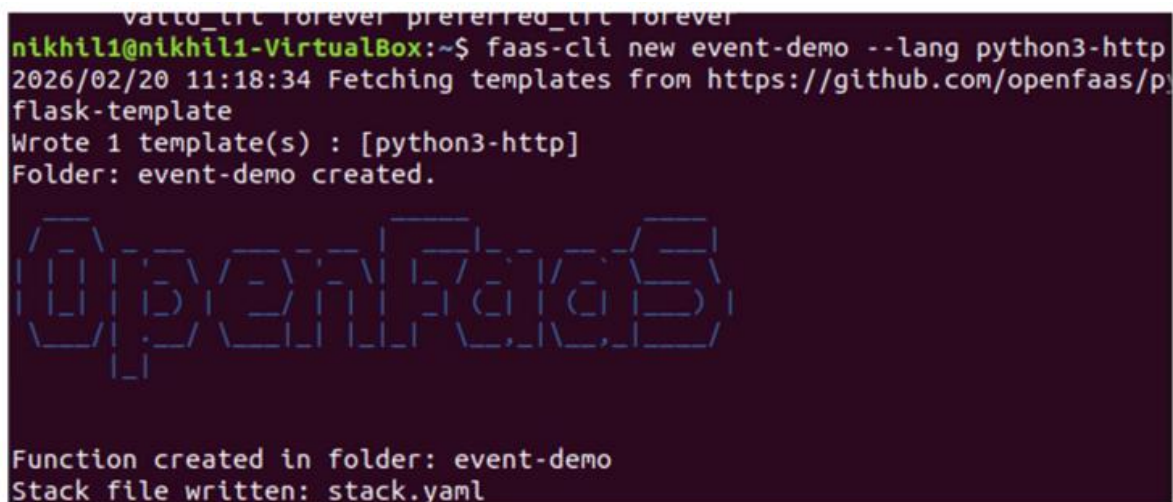
sudo cat /var/lib/faasd/secrets/basic-auth-password

The above command gives a password paste that password in below command

export OPENFAAS_URL=http://127.0.0.1:8080

echo -n "YOUR-PASSWORD" | faas-cli login --username admin --password-stdin

faas-cli new event-demo --lang python3-http

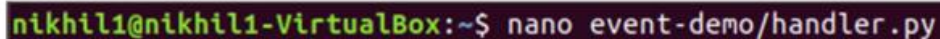


```
vattu_tit forever preferred_tit forever
nikhil1@nikhil1-VirtualBox:~$ faas-cli new event-demo --lang python3-http
2026/02/20 11:18:34 Fetching templates from https://github.com/openfaas/p
flask-template
Wrote 1 template(s) : [python3-http]
Folder: event-demo created.

OpenFaaS

Function created in folder: event-demo
Stack file written: stack.yaml
```

nano event-demo/handler.py



```
nikhil1@nikhil1-VirtualBox:~$ nano event-demo/handler.py
```

replace code with

```
<<from flask import request
```

```
def handle(event, context):
```

```
    return "Event processed successfully">>
```

Save:

- CTRL + O
- Enter
- CTRL + X

nano stack.yaml

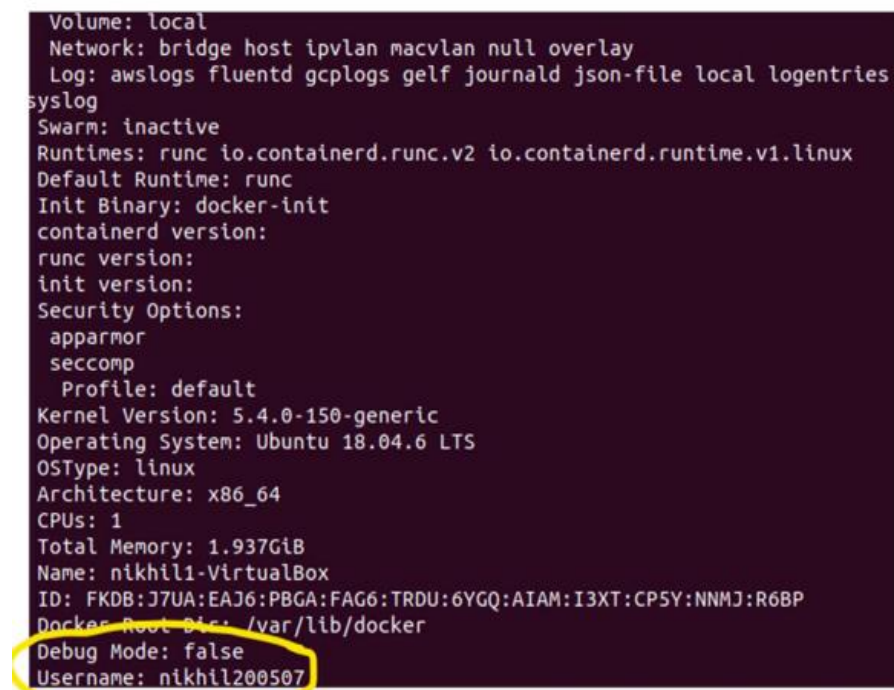
find

image: yourdockerhubusername/event-demo:latest or event-demo:latest

place your dockerhub username

you can find dockerhub username by typing command

docker info



```
Volume: local
Network: bridge host ipvlan macvlan null overlay
Log: awslogs fluentd gcplogs gelf journald json-file local logentries
syslog
Swarm: inactive
Runtimes: runc io.containerd.runc.v2 io.containerd.runtime.v1.linux
Default Runtime: runc
Init Binary: docker-init
containerd version:
runc version:
init version:
Security Options:
  apparmor
  seccomp
  Profile: default
Kernel Version: 5.4.0-150-generic
Operating System: Ubuntu 18.04.6 LTS
OSType: linux
Architecture: x86_64
CPUs: 1
Total Memory: 1.937GiB
Name: nikhil1-VirtualBox
ID: FKDB:J7UA:EAJ6:PBGA:FAG6:TRDU:6YGQ:AIAM:I3XT:CP5Y:NNMJ:R6BP
Docker Root Dir: /var/lib/docker
Debug Mode: false
Username: nikhil200507
```

save and close

faas-cli build --no-cache

```

nikhil@nikhil-VirtualBox:~/temp$ faas-cli build --no-cache
[0] > Building event-demo.
Building: nikhil200507/event-demo:latest with python3-http template. Please
t..
2026/02/20 11:56:38 Build flags: [build --no-cache --build-arg ADDITIONAL_P
GE= --tag nikhil200507/event-demo:latest .]
Sending build context to Docker daemon 13.82kB
Step 1/38 : ARG PYTHON_VERSION=3.12
Step 2/38 : FROM --platform=${TARGETPLATFORM:-linux/amd64} ghcr.io/openfaas
watchdog:0.11.3 AS watchdog
0.11.3: Pulling from openfaas/of-watchdog
c509d8994223: Pulling fs layer
c509d8994223: Verifying Checksum
c509d8994223: Download complete
c509d8994223: Pull complete
Digest: sha256:853a3654aeb052951a14aa13dd050d894a5b65ecb6ad3ae79eeeb453a7cf
Status: Downloaded newer image for ghcr.io/openfaas/of-watchdog:0.11.3
--> fc6c830af188
Step 3/38 : FROM --platform=${TARGETPLATFORM:-linux/amd64} python:${PYTHON_
ION}-alpine AS build
3.12-alpine: Pulling from library/python
589002ba0eae: Pulling fs layer
aca07fe076f1: Pulling fs layer
4e18f69996e7: Pulling fs layer
77e59d578b52: Pulling fs layer
77e59d578b52: Waiting
aca07fe076f1: Verifying Checksum

```

faas-cli push

```

nikhil@nikhil-VirtualBox:~/temp$ faas-cli push
[0] > Pushing event-demo [nikhil200507/event-demo:latest]
The push refers to repository [docker.io/nikhil200507/event-demo]
4009b52492b7: Pushed
c6e778a4a1d3: Pushed
b4ebf65d4add: Pushed
a12827768cd8: Pushed
c7e1ecf56fa5: Pushed
3db1ee46e09a: Pushed
e945883b18c6: Pushed
043b04ef6f41: Pushed
cebf294fc92c: Pushed
2ef3d129c542: Pushed
f8332ce2bf4d: Pushed
96cba8cd2d37: Pushed
305efd2d25f2: Mounted from library/python
45141c3f736e: Mounted from library/python
f48db1250229: Mounted from library/python
989e799e6349: Mounted from library/python
latest: digest: sha256:d08970d1e8b15b3fdc12c660ce12e5732674ee245fd72ffacbf7d
3220f6ff size: 3863
[0] < Pushing event-demo [nikhil200507/event-demo:latest] done.
[0] Worker done

```

faas-cli deploy

```

nikhil@nikhil-VirtualBox:~/temp$ faas-cli deploy
Deploying: event-demo.

Deployed. 200 OK.
URL: http://127.0.0.1:8080/function/event-demo

```

PASS=\$(sudo cat /var/lib/faasd/secrets/basic-auth-password)

curl -u admin:\$PASS http://127.0.0.1:8080/function/event-demo -d "Hello"

```

nikhil@nikhil-VirtualBox:~/temp$ PASS=$(sudo cat /var/lib/faasd/secrets/basic
-auth-password)
nikhil@nikhil-VirtualBox:~/temp$ curl -u admin:$PASS http://127.0.0.1:8080/fu
nction/event-demo -d "hello"
event processed successfully
nikhil@nikhil-VirtualBox:~/temp$

```

Exp – 13

Aim

To simulate a cloud scenario using CloudSim and execute a scheduling algorithm.

CloudSim is a simulation toolkit used to model and simulate cloud computing environments. It helps in analyzing resource allocation, scheduling algorithms, and performance without using real cloud infrastructure.

Procedure

1. Install Java (JDK 1.8).
2. Set Java environment variables.
3. Download and install Eclipse IDE.
4. Download CloudSim package and extract it.
5. Download required jar files (commons-math3).
6. Add all jar files to Eclipse project.
7. Create a new Java project.
8. Create a new class inside src folder.
9. Write CloudSim simulation program.
10. Run the program in Eclipse.

Program

```
package cloudsimproject;

import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;
import java.util.*;

public class Test {
    public static void main(String[] args) throws Exception {

        CloudSim.init(1, Calendar.getInstance(), false);

        Pe pe = new Pe(0, new PeProvisionerSimple(1000));

        Host host = new Host(0,
            new RamProvisionerSimple(2048),
```



```

        new BwProvisionerSimple(10000),
        1000000,
        Arrays.asList(pe),
        new VmSchedulerSpaceShared(Arrays.asList(pe)));

    Datacenter dc = new Datacenter("DC",
        new DatacenterCharacteristics("x86", "Linux", "Xen",
            Arrays.asList(host), 10, 3, 0.05, 0.1, 0),
        new VmAllocationPolicySimple(Arrays.asList(host)),
        new LinkedList<Storage>(), 0);

    DatacenterBroker broker = new DatacenterBroker("Broker");

    CloudSim.startSimulation();
    CloudSim.stopSimulation();
}
}

```

Output

CloudSim simulation runs successfully and displays execution details in console.

Additional Programs

1) Dockerized Student Profile Website

Description: Creating a personal profile webpage using HTML and CSS, containerize it using Docker, and publish the Docker image to Docker Hub.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dev</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h2>Aradhyula Vasu Dev Manikanta Sai Nadh</h2>
  </header>
  <section class="hero">
    <div class="hero-text">
      <h3>B.Tech CSE Student</h3>
      <p>
        I am a Computer Science Engineering student with a strong interest in
        software development and modern technologies. I enjoy understanding how
        systems work and applying my knowledge to solve real-world problems.
        My academic journey has helped me build a solid foundation in programming
        and analytical thinking.
      </p>
      <p>
        I actively explore areas like web development, cloud computing, and
        emerging technologies to improve my technical skills. I believe in
        continuous learning and enjoy working on practical projects that
        strengthen my understanding and experience.
      </p>
      <p>
        My goal is to become a skilled software developer and contribute to
```

innovative and impactful solutions. I am committed to improving my abilities, adapting to new technologies, and growing in a professional environment.

</p>

</div>

<div class="hero-img">

<div class="side-menu">

Home

About

Contact

</div>

</div>

</section>

<section class="section card-section">

<h2>Education</h2>

<div class="card">

<h3>Vishnu Institute of Technology</h3>

<p>B.Tech in Computer Science Engineering</p>

<p>CGPA: 9.01 (2023–2027)</p>

</div>

</section>

<section id="skills" class="section">

<h2>Technical Skills</h2>

<div class="skills">

Python

Java

C

JavaScript

HTML

CSS

SQL

MongoDB

Git

</div>

Cloud Computing Lab

Page No...

```
</section>
<section class="section card-section">
  <h2>Experience</h2>
  <div class="card">
    <h3>AI-ML Virtual Internship</h3>
    <ul>
      <li>Learned practical AI & ML concepts</li>
      <li>Implemented basic machine learning models</li>
      <li>Worked with datasets and analysis techniques</li>
    </ul>
  </div>
</section>
<section id="projects" class="section card-section">
  <h2>Projects</h2>
  <div class="card">
    <h3>BinSol – IoT Monitoring System</h3>
    <ul>
      <li>Developed real-time garbage bin monitoring system</li>
      <li>Used sensors for automated alerts</li>
      <li>Improved waste management efficiency</li>
    </ul>
  </div>
</section>
<section class="section">
  <h2>Certifications</h2>
  <ul class="cert">
    <li>AI-ML Virtual Internship</li>
    <li>Privacy & Security in Online Social Media</li>
    <li>Full Stack Web Application Development</li>
  </ul>
</section>
</body>
</html>
```

CSS

body {

Cloud Computing Lab

Page No...

```
margin: 0;
font-family: 'Segoe UI', sans-serif;
background: #f4f6f9;
color: #333;
}
/* HEADER */
header {
display: flex;
justify-content: center;
padding: 20px;
background: white;
box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
header h2 {
color: black;
font-size: 26px;
font-weight: bold;
}
/* HERO */
.hero {
display: flex;
justify-content: space-between;
align-items: center;
padding: 60px;
}
/* TEXT */
.hero-text {
width: 60%;
background: linear-gradient(135deg, #4a00e0, #8e2de2);
color: white;
padding: 20px;
border-radius: 10px;
}
.hero-text h3 {
font-size: 24px;
```

```
margin-bottom: 10px;
}
.hero-text p {
  line-height: 1.7;
  margin-top: 10px;
}
/* IMAGE SIDE */
.hero-img {
  width: 35%;
  display: flex;
  flex-direction: column;
  align-items: center;
}
/* IMAGE */
.hero-img img {
  width: 220px;
  height: 220px;
  border-radius: 50%;
  border: 5px solid white;
  object-fit: contain;
  background: #e6e6e6;
  padding: 20px;
}
/* MENU */
.side-menu {
  margin-top: 10px;
  display: flex;
  gap: 15px;
}
.side-menu a {
  text-decoration: none;
  color: #4a00e0;
  font-weight: 500;
}
/* SECTION */
```

```
.section {
padding: 40px 60px;
}
.section h2 {
color: #4a00e0;
margin-bottom: 15px;
}
/* CARDS */
.card-section {
background: #fff;
}
.card {
background: white;
padding: 20px;
border-radius: 10px;
box-shadow: 0 10px 20px rgba(0,0,0,0.1);
margin-top: 10px;
}
/* SKILLS */
.skills {
display: flex;
flex-wrap: wrap;
gap: 10px;
}
.skills span {
background: #4a00e0;
color: white;
padding: 8px 15px;
border-radius: 20px;
font-size: 14px;
}
/* LIST */
ul {
padding-left: 20px;
}
```

```
.cert li {  
  margin: 5px 0;  
}
```

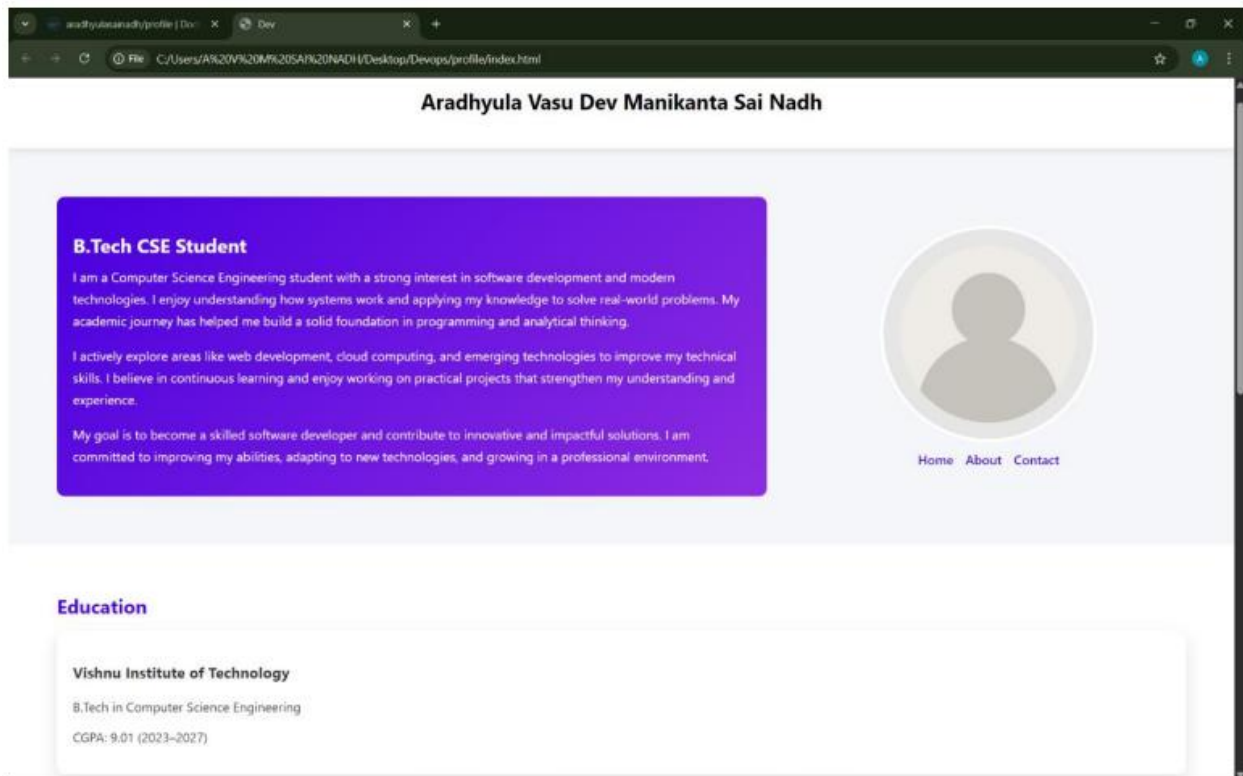
Dockerfile

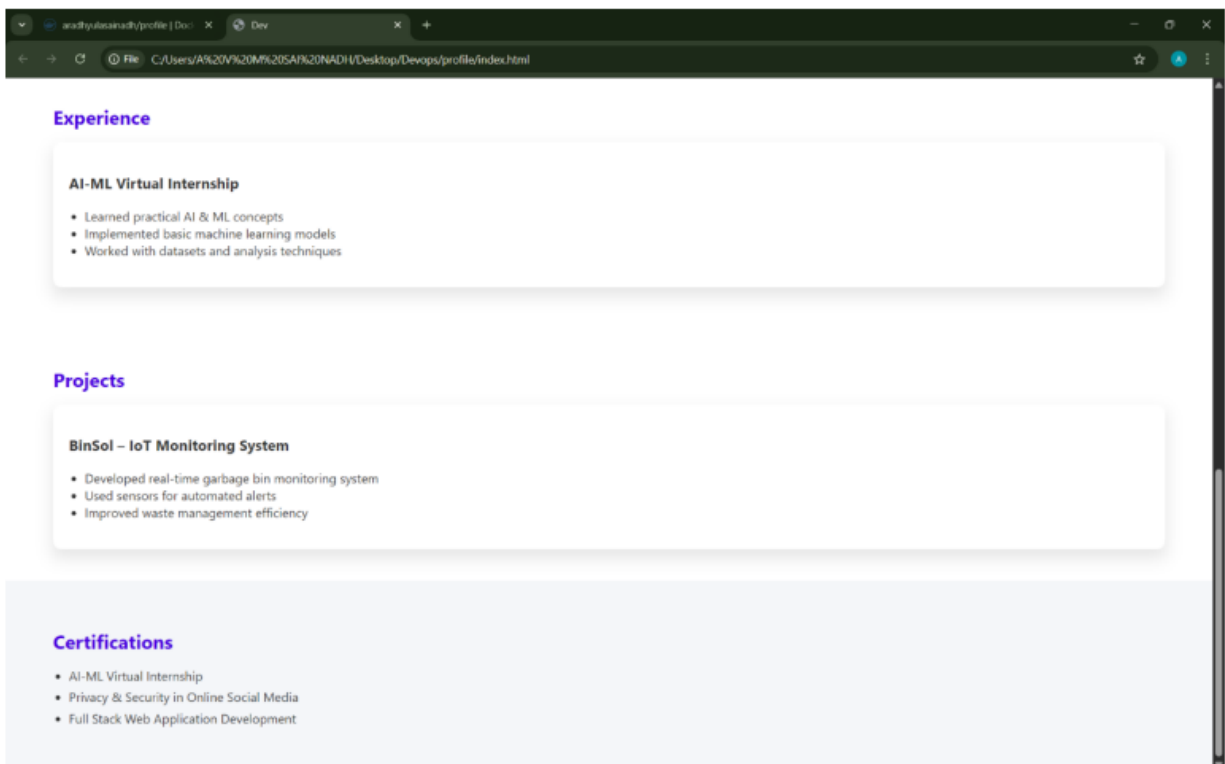
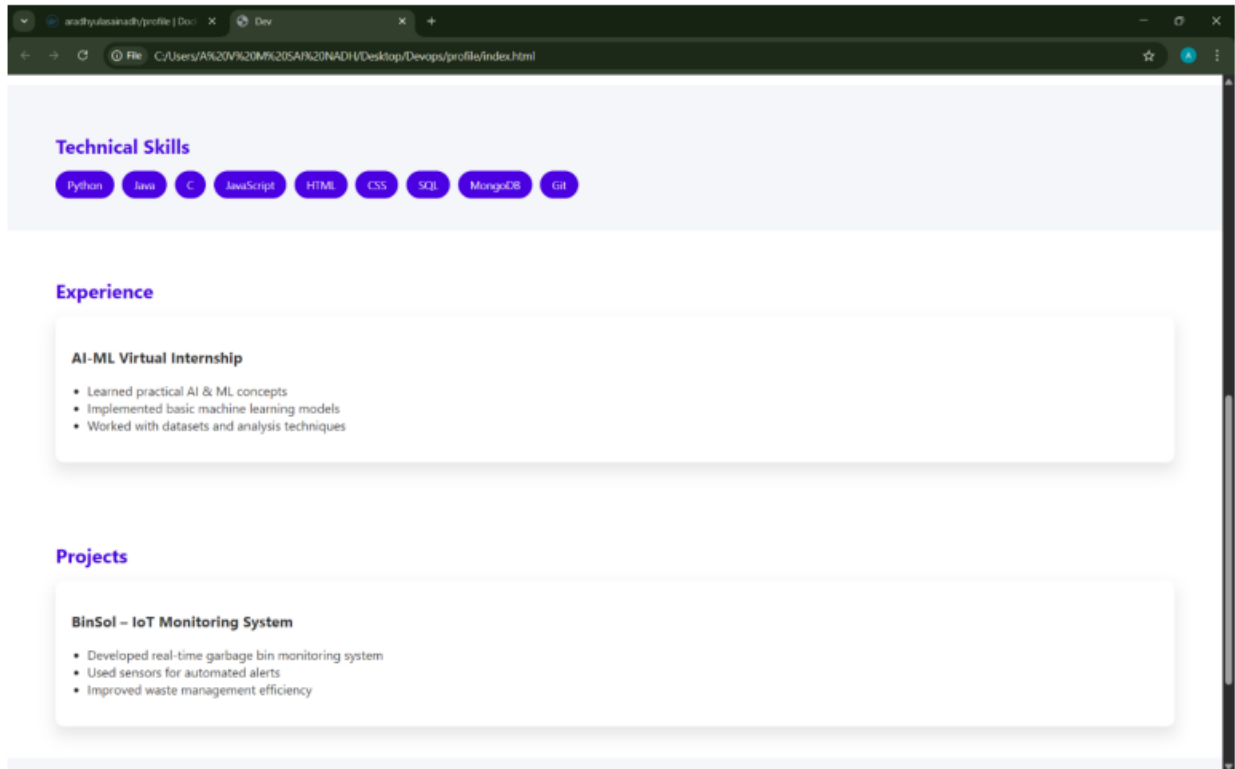
FROM nginx:alpine

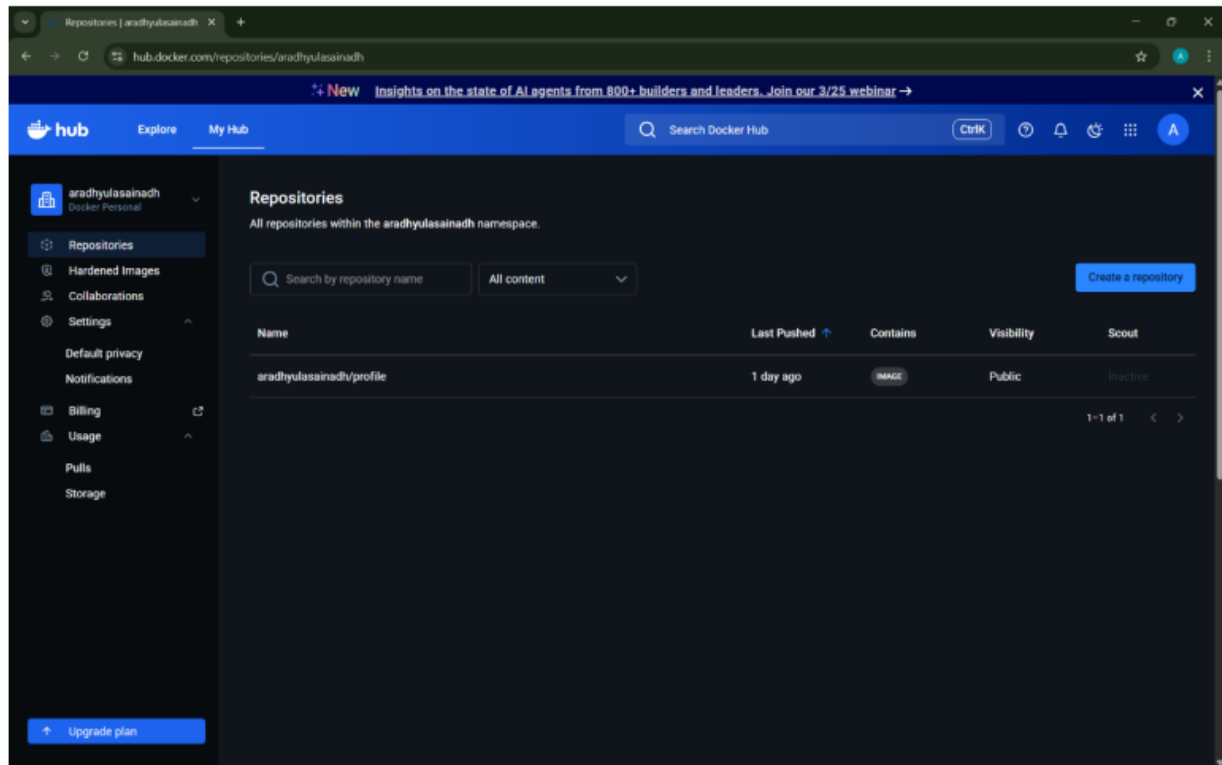
COPY . /usr/share/nginx/html

EXPOSE 80

IMAGES







2) Basic Web Automation Using Selenium

Description: Selenium WebDriver to automate basic browser actions. opening websites, locating elements, entering text, checking page content, and using explicit waits

Code_JavaScript

```
const { Builder, By, Key, until } = require('selenium-webdriver');
const chrome = require('selenium-webdriver/chrome');
async function runTest() {
  let options = new chrome.Options();
  options.addArguments('--log-level=3');
  options.addArguments('--silent');
  let browser = await new Builder()
    .forBrowser('chrome')
    .setChromeOptions(options)
    .build();
  try {
    const timeout = 10000;
    await browser.get("https://example.com");
    let title = await browser.getTitle();
    console.log("Page Title is:", title);
    await browser.get("https://www.wikipedia.org");
    let searchBox = await browser.wait(until.elementLocated(By.name("search")), timeout);
    await searchBox.sendKeys("A V M SAI NADH", Key.ENTER);

    await browser.wait(until.titleContains("Search results"), timeout);
    await browser.get("https://example.com");
    let headerElement = await browser.wait(until.elementLocated(By.tagName("h1")), timeout);
    let text = await headerElement.getText();
    if (text === "Example Domain") {
      console.log("Correct Heading Found");
    } else {
      console.log("Heading Missing");
    }
    console.log("Loaded Heading:", text);
  }
}
```

```
} catch (error) {  
  console.error("Error:", error.message);  
} finally {  
  await browser.quit();  
}  
}
```

IMAGES

